COMP8410 2026 S2 — Assignment 1: Evaluation of Seven Pairs of Generative AI Models for a National Online Property Marketplace

Yash Ram Nimbalkar (Student ID 48576158)

8th November 2026



Contents

1 Introduction and scenario ... 3
1.1 The organisation ... 3
1.2 Evidence status of the scenario's premises ... 3
1.3 What the scenario implies for model selection ... 4
1.4 The seven pairs ... 4
1.5 How to read this report ... 5
2 Shared evaluation methodology ... 6
2.1 The evaluation framework ... 6
2.2 Controlled comparison protocol ... 6
2.3 Items and replication ... 7
2.4 Metric families ... 7
2.5 Statistical treatment ... 7
2.6 Metric validity controls ... 7
2.7 Qualitative inspection ... 8
2.8 Reproducibility and evidence ... 8
3 Pair 1 — Instruction-driven image editing ... 9
3.1 Selection ... 9
3.2 Evaluation ... 10
4 Pair 2 — Identity-preserving portrait generation ... 15
4.1 Selection ... 15
4.2 Evaluation ... 16
5 Pair 3 — Walkthrough reconstruction from a photo set ... 21
5.1 Selection ... 21
5.2 Evaluation ... 22
6 Pair 4 — Single photo to 3D asset ... 26
6.1 Selection ... 26
6.2 Evaluation ... 27
7 Pair 5 — Listing photo to short motion clip ... 32
7.1 Selection ... 32
7.2 Evaluation ... 33
8 Pair 6 — Editing an existing walkthrough clip ... 37
8.1 Selection ... 37
8.2 Evaluation ... 38
9 Pair 7 — Agents that operate the listing web UI ... 42
9.1 Selection ... 42
9.2 Evaluation ... 43
10 Cross-pair synthesis ... 47
10.1 Only one pair produced a clean winner, and it is the only pair with ground truth ... 47
10.2 Licensing lives in the dependency tree, and it decided more pairs than quality did ... 47
10.3 Wall-clock speed advantages are usually properties of sampling schedules ... 48

10.4 Preservation metrics reward models that do less....48
10.5 Metric validity controls paid for themselves....49
10.6 Aggregates concealed the structure that mattered....49
10.7 Harness defects are indistinguishable from model results until excluded....49
10.8 A consistent gap: no non-learned baselines were run....50
10.9 Recommendations for the marketplace....50
10.10 Limitations of this evaluation....51
11 Compute budget and cost....52
11.1 Recorded and reconstructed spend....52
11.2 The budget constrained the evaluation more than the spend justifies....52
12 User feedback....53
12.1 What exists....53
12.2 What does not exist....53
12.3 Why this matters more than a missing nice-to-have....53
13 References....54

1 Introduction and scenario

1.1 The organisation

The system under evaluation is a **national online property marketplace**: a two-sided platform where real-estate agencies and private sellers post residential listings, while buyers and renters browse, shortlist and make enquiries. What sets one listing apart from another is its media — photographs, walkthroughs and floor plans.

This organisation is hypothetical. It is a composite scenario built for this assignment rather than a real, named company, and nothing here claims that any actual marketplace works in the way described. Wherever the scenario depends on a factual claim about the industry, that claim is either supported by evidence below or explicitly flagged as an assumption.

1.2 Evidence status of the scenario's premises

The rubric calls for selection that is both justified and critical. A number of premises that would normally be taken for granted in this kind of scenario do not hold up under scrutiny, so they are set out separately here instead of being allowed to quietly shape the rest of the report.

Premise 1: the marketplace operates at high volume. *Assumption, not evidenced.* The quantities used throughout this report — roughly tens of thousands of listings each month, with ten to forty photographs apiece — are stipulated by the scenario so that throughput and unit cost become meaningful constraints. No public dataset could be found that substantiates them for any particular marketplace. They should be understood as "the regime this evaluation targets" rather than as a measured fact. The evaluation design does not hinge on their precise values; what matters is that per-item cost is a binding constraint, which is true of any deployment at batch scale.

Premise 2: better listing media produces better listing performance. *Partly contradicted by the available evidence.* This is the assumption on which the whole project would ordinarily rest, and it fails in the simple form in which it is usually expressed. In a study of 3,658 Paris apartment advertisements, Larceneux, Bezançon and Lefebvre found that **increasing the number of photographs reduced click rates for lower-range properties** (below roughly €10,331/m²) and had no significant effect for higher-range ones; in laboratory experiments, adding non-informative photographs did not raise intention to click, whereas photographs carrying negative information lowered it. Their study measured photograph *count* and deliberately did not control for photograph *quality*.

Two consequences follow for this report. First, the commercial justification for the capabilities assessed here cannot be built on "more media is better", since the single empirical study found points the opposite way for a sizeable market segment. Second, the quantity-versus-quality distinction becomes the critical one: the evidence is compatible with a world where extra uninformative media is neutral or damaging while better media is valuable, but **that is an inference, not a finding**, and no study establishing it could be located. The user survey accompanying this assignment is the only direct evidence of human preference gathered here, and it is small.

Premise 3: misrepresenting a property through edited media carries regulatory exposure. *Evidenced, with qualifications.* In New South Wales, the *Residential Tenancies (Protection of Personal Information) Amendment Bill 2025* would oblige landlords and agents to disclose when images have been altered to hide faults or to mislead applicants — the government's own worked example is artificially generated furniture that makes a room look bigger than it really is — with penalties of $5,500 for individuals and $22,000 for businesses. Three qualifications matter: it is **a bill, not yet law**, with no stated commencement date; it covers **residential tenancy** advertising rather than sales; and it is **one state**. Statements in this report about regulatory exposure are therefore statements about a clearly signalled direction of travel, not about an established national obligation. Separately, NSW Fair Trading has received

reports of listing images being digitally altered to conceal damage, and the Tenants' Union has said that digital manipulation in rental advertising has increased; neither source offers systematic prevalence data.

Premise 4: marketplaces are deploying generative models of the kind evaluated here. *No public evidence of a specific pairing was located.* Searches for a named property marketplace publicly using any of the fourteen models assessed in this report — via press release, case study or job posting — found nothing that names a specific model and a specific platform together. Industry commentary portrays real-estate agents as early adopters of AI tooling in general, and a virtual-staging vendor market clearly exists, but that does not show that these models are in production at any marketplace. **This report therefore makes no claim that its scenario reflects current industry practice.** It asks whether these models *could* meet the stated capability needs, which is a different — and answerable — question.

1.3 What the scenario implies for model selection

Four properties of the setting shape the evaluation design. The first three follow from the premises above; the fourth is simply a property of user-generated media.

1. **Volume makes cost a first-class criterion.** At batch scale, throughput, latency and peak VRAM decide whether a capability can be deployed at all, regardless of output quality. That is why cost metrics appear in every pair and are weighed alongside quality instead of being pushed into a footnote.
2. **Fidelity is a constraint, not a preference.** Given the regulatory direction in Premise 3, a model that invents or removes a material feature of a property creates a misrepresentation risk rather than a mere aesthetic flaw. Every pair that edits or synthesises imagery therefore measures *preservation of what was not asked to change* separately from *execution of what was asked*, and the two are never merged into a single quality score.
3. **Quality must be distinguished from quantity.** Following Premise 2, the evaluation asks whether a model produces media that is *better*, not merely *more*, and treats instruction execution and visual fidelity as separate measurements.
4. **Inputs are heterogeneous.** Photographs and clips arrive from phones and professional shoots, under varying light and at varying resolutions. Robustness across that spread counts for more than peak performance on clean inputs, which is why the item sets in this report are drawn from realistic rather than curated material — including, in Pair 6, hand-held walkthrough footage shot specifically for the purpose.

1.4 The seven pairs

Each pair answers one capability question for this marketplace. In every case, two current models are compared head-to-head on identical inputs, seeds and hardware.

#	Capability for the marketplace	Model A	Model B
1	Instruction-driven photo editing (relighting, time-of-day, weather)	FLUX.1 Kontext [dev]	Qwen-Image-Edit-2509
2	Identity-preserving portraits for agent profiles	InstantID	PuLID v1.1
3	Walkthrough reconstruction from a photo set	Nerfacto (NeRF)	Splatfacto (3DGS)
4	Single-photo to 3D asset for furnishing/staging	TRELLIS2	Hunyuan3D
5	Listing photo to short motion clip	Wan2.2 TI2V-5B	HunyuanVideo 1.5 (480p I2V)
6	Editing an existing walkthrough clip	Wan2.1-VACE-1.3B	Lucy-Edit-Dev
7	Agents that operate the listing web UI	Qwen2.5-VL-7B (DOM text)	UI-TARS-1.5-7B (screenshots)

1.5 How to read this report

Every pair chapter follows the same structure: **Selection** (the modelling technique and the modelling context for the scenario) followed by **Evaluation** (methodology, results, reflection). The shared methodology set out in the next chapter applies to all pairs; pair chapters record only their deviations from it.

Two conventions are used throughout. A difference between two models is reported as a **paired bootstrap over items** and is called a difference only when the interval excludes zero — otherwise the recorded verdict is "no clear difference", which happens often and is reported as a result in its own right rather than being resolved by picking the larger mean. In addition, every claim that traces back to a measurement cites the file that produced it, so the tables in this report can be regenerated from the accompanying code.

2 Shared evaluation methodology

All seven pairs are evaluated under one framework, so that conclusions remain comparable across capabilities as different as mesh generation and GUI automation. This chapter defines that framework once; each pair chapter records only where it departs from it.

2.1 The evaluation framework

The framework is built around six dimensions. The first four are measured directly from the runs; the fifth is reasoned from those measurements; the sixth is gathered separately from people. Keeping them as distinct dimensions is deliberate — a recurring finding of this report is that a model can perform well on one dimension while failing another, and a single blended "quality" score would hide precisely the trade-offs on which a deployment decision turns.

#	Dimension	How it is measured here	Where reported
1	Role and appropriateness against current practice	What the model family is for, what it superseded, whether the capability matches the scenario's stated need	Selection, each pair
2	Technical performance	Task execution and fidelity metrics, with paired intervals over items	Results, each pair
3	Business performance	Latency, throughput, peak VRAM, artefact size, compute cost in A$	Results, each pair
4	Competitiveness and uniqueness	Head-to-head paired difference, plus a non-learned or naive baseline where one exists	Results, each pair
5	Social, economic and environmental consequence	Disclosure and misrepresentation exposure, consent and likeness, compute footprint	Modelling context and Reflection, each pair
6	User feedback	A/B preference survey on paired outputs	User feedback chapter

Dimension 3 carries the same weight as dimension 2 throughout. This follows from Premise 1 of the scenario: at batch scale, a capability that cannot be afforded is not really a capability, so a model that is marginally better but three times more expensive has not necessarily won.

2.2 Controlled comparison protocol

For every pair, both models receive **identical inputs, identical prompts, identical seeds and identical hardware**. Items are prepared once, written to a working directory, and read by both models from the same files; no per-model preprocessing takes place.

Where a setting cannot be equalised — because the two models do not expose the same control, or because one does not fit in memory at the other's settings — the comparison is declared **system-level** rather than model-level, and the specific confound is named in the relevant chapter. The distinction is not cosmetic. A system-level result states what the marketplace would obtain by deploying each stack as published; it does not support any claim that the architecture caused the difference. Pair 5 is system-level for this reason, and no architectural conclusion is drawn from it.

Two asymmetries that could not be eliminated are declared where they arise rather than hidden: the mask input available to one model but not the other in Pair 6, and the single round of prompt iteration that one agent received in Pair 7.

2.3 Items and replication

Each pair works from a fixed item set drawn from the scenario — listing photographs, agent portraits, captured rooms, furnishing objects, walkthrough clips or benchmark tasks — enumerated in that pair's items.csv or config.yaml. Item counts range from 6 (the two video pairs, where one run costs minutes of accelerator time) up to 48 item-seed combinations (identity preservation).

Where generation is stochastic, each item is run on multiple fixed seeds. This is reported as measured rather than assumed: in Pair 4 the within-item standard deviation across seeds is exactly 0.000, which means the pipelines are deterministic at those settings and the seed replication contributes no variance information whatsoever. Pairs 5 and 6 use a single seed as a budget decision. In those three pairs, **item-level variation is the only source of uncertainty in the reported intervals**, and the intervals should be read with that in mind.

2.4 Metric families

Three families of metrics are reported for every pair.

Task metrics ask whether the model did what was asked — a CLIP text-image score for instruction adherence, binary task success for agents, or similarity to a held-out ground truth where one exists.

Fidelity metrics ask whether the model preserved what it was *not* asked to change — structural similarity on edge maps, image-embedding similarity to the source, PSNR and SSIM restricted to the region outside an edit, silhouette agreement, or face-embedding similarity. Separating these from task metrics follows directly from the fidelity constraint in the Introduction: in this scenario an unrequested change is a compliance problem, not a stylistic one, so it has to be measured on its own and never averaged into a single score.

Cost metrics — latency, peak VRAM, throughput, artefact size and, where measurable, A$ per unit of output — are reported with the same prominence as the other two families.

2.5 Statistical treatment

Differences between the two models in a pair are reported as a **paired bootstrap over items** on the per-item difference, with a 95% percentile interval, rather than as a comparison of two independent means. Pairing removes item difficulty as a source of variance, which matters at these sample sizes: the same room is hard for both models, so comparing them on that same room is far more informative than comparing two averages.

A difference is counted as a difference **only when the interval excludes zero**. Otherwise the recorded verdict is **"no clear difference"**. That verdict is common in this report and is treated as a finding, not resolved by choosing the larger mean. Where a non-significant difference still looks suggestive, the per-item win count is given so the reader can judge direction and consistency independently of the interval.

Per-model central values are reported with their own bootstrap intervals, and the two should not be compared by eye: overlapping per-model intervals are entirely consistent with a significant paired difference, and the reverse is also true.

2.6 Metric validity controls

A similarity metric is only informative if it can tell matched content from mismatched content. Wherever a pair relies on one, a **mismatch control** is run: the same metric is computed between each output and a *different* item's source or instruction, and the gap between the matched and mismatched scores is reported as the metric's usable range.

This is not a formality. In Pair 4 the control confirms that DINO similarity discriminates strongly (a gap of about 0.68) while CLIP image similarity discriminates weakly (0.19), which dictates how much weight each result can bear. In Pair 6 the control shows the matched-versus-mismatched gap for CLIP text similarity is only 0.036, so the entire measured edit effect sits within the metric's own noise floor — and indeed that metric went on to rank the model that visibly failed the task above the one that visibly succeeded. Had the control not been run, that inversion would have been reported as a result.

Wherever two independent backbones are available for the same construct, both are used. In Pair 2 this is what exposed a confound: a large identity advantage under one face-recognition backbone fails to replicate under a second, and the difference tracks how large the model renders the face rather than how faithful the identity is.

2.7 Qualitative inspection

Every pair that produces an image, mesh or clip is inspected visually before its numbers are interpreted, using contact sheets and frame strips produced by the reporting scripts and kept as evidence. Inputs are likewise inspected before any accelerator time is spent, after an early run in this project burned a full generation pass on the wrong source images.

Inspection is a mandatory step of the methodology, not a courtesy, and Pair 6 is the justification. There, **every automatic metric identified the wrong winner**, and the correct result could only be recovered by looking at the frames. A framework that simply trusted its metrics would have reported the opposite conclusion, with confidence intervals attached.

2.8 Reproducibility and evidence

Every number in this report is produced by a script in the accompanying 48576158 Evaluation folder and appended to a per-pair JSON-lines results file — one row per measurement of one metric for one model on one item. Long-format rows allow a single aggregation module to generate comparable tables for all fourteen models, and the append-only file doubles as a record of the run.

Report tables and figures are regenerated from those files by the per-pair report_*.py scripts, so no figure in this document has been assembled by hand. Alongside the results, the repository keeps per-pair environment captures (pip freeze), upstream repository commit hashes for models installed from source, run logs, and the generation sidecar files recording latency and peak memory for each item. Pure functions in the scoring and generation code are covered by unit tests that run on CPU before any accelerator time is spent; these caught three defects in Pair 6's metrics that would otherwise have yielded empty or misleading columns.

Where a defect was discovered after a run, the superseded results file is kept rather than deleted — results/w7_vace_conditioning_bug.jsonl preserves the first, incorrect pass of Pair 6 so that the correction can be audited. See docs/EVIDENCE.md for the full inventory.

3 Pair 1 — Instruction-driven image editing

FLUX.1 Kontext [dev] vs Qwen-Image-Edit-2509

3.1 Selection

3.1.1 AI modelling techniques

Both models carry out **instruction-conditioned image-to-image editing**: given a source photograph and a natural-language instruction, they return an edited photograph in which the instruction has been applied while everything else is meant to stay unchanged. For the marketplace, the use case is normalising listing photography that arrives in poor or inconsistent light — turning a flat, overcast interior shot into warm evening light, for instance — without organising a reshoot.

Both belong to the **diffusion-transformer** generation of editors rather than the U-Net architectures that came before them, and both accept the source image as *in-context* conditioning: the image is encoded into the same latent sequence over which the transformer attends, instead of being injected through a separate adapter or supplied as a masked region. This property is what makes them usable by a non-expert operator. Earlier editing pipelines required the user to draw a mask marking what should change, or to fine-tune per subject; in-context editing reduces the interface to a sentence, which matters when the operator is a sales agent rather than a retoucher.

They differ in three respects that matter for the evaluation:

	FLUX.1 Kontext [dev]	Qwen-Image-Edit-2509
Developer	Black Forest Labs	Alibaba (Qwen)
Parameters	12 B	20 B
Formulation	Rectified flow transformer, guidance-distilled	MMDiT (multimodal diffusion transformer)
Licence	FLUX.1 [dev] **Non-Commercial** Licence	**Apache 2.0**

The parameter gap is close to two-fold, so this is **not a matched-size comparison**. Any speed or memory advantage FLUX shows is therefore partly the advantage of being the smaller model, and it is decomposed rather than attributed to architecture in the Results.

These in-context editors are the current endpoint of the arc covered in Week 2's image-editing material, which worked through attention-level control — visualising cross-attention maps, Prompt-to-Prompt-style editing by manipulating them, and self-attention segmentation. That lineage requires access to the model's internals and per-edit prompt engineering; in-context conditioning supersedes it by reducing the interface to a sentence, which is precisely the property a non-expert operator needs. [Confirm against the Week 2 deck.]

3.1.2 Modelling context

Item	FLUX.1 Kontext [dev]	Qwen-Image-Edit-2509
Objectives	Normalise listing photography at low marginal cost; maximise photos processed per GPU-hour	Same business objective, with structural fidelity prioritised over throughput
Tasks	Relighting, time-of-day and weather transfer on interior and exterior stills	Identical task set (the evaluation holds the task fixed)
Functions	Single-pass edit from one instruction; no mask, no per-item tuning	Same, plus multi-image input (unused here)
Framework	12 B rectified-flow transformer over a latent image sequence; source image enters as in-context tokens; guidance distilled, so it runs at low CFG (2.5) and few steps (28)	20 B MMDiT; source image and instruction jointly conditioned; run at CFG 4.0 and 40 steps

Algorithms	Flow matching in latent space (the model card's own citation is to flow matching for in-context image generation and editing)	Multimodal diffusion transformer denoising
Governance	**Non-commercial licence** — blocking for this scenario without a paid agreement. Licence also obliges deployers to run content filters or manual review, and Black Forest Labs states it may contact deployers to verify this	**Apache 2.0** — no licensing obstacle to commercial deployment, and no vendor-imposed filtering obligation (which shifts the content-safety duty entirely onto the marketplace)
Advantages	Markedly faster and lighter; fits a single 40 GB accelerator with room to spare	Better structural preservation (measured below); permissive licence; larger model with headroom for harder edits

The licensing difference is the single most consequential item in this table, and it is invisible in every quality metric. FLUX.1 Kontext [dev] is distributed under the FLUX.1 [dev] Non-Commercial Licence. A national marketplace using it to process paying customers' listings would be making commercial use of the weights and would need a separate commercial agreement with Black Forest Labs. Qwen-Image-Edit-2509 is Apache 2.0 and carries no such restriction.

Two qualifications are owed here. First, the FLUX model card is internally inconsistent: its feature list says generated outputs may be used commercially while the licence it names is a non-commercial one — the usual distinction between *outputs* and *weights*, but one the card does not spell out. **The licence text itself should be read before any deployment decision; this report does not resolve the ambiguity.** Second, the card consulted was a public mirror rather than the official Black Forest Labs repository, which could not be reached from the evaluation environment — so the licence name is reported with that provenance noted.

3.2 Evaluation

3.2.1 Methodology

Items. 15 listing photographs captured for this assignment, resized and written to data/processed/w2_listing/ by prepare_items.py, with the item table in items.csv. The edits span four families, deliberately chosen to cover a range of edit magnitudes:

Edit family	Photos	Instruction (abbreviated)
brighten	3	brighten the room, keep everything else identical
overcast	3	soft overcast day, even diffuse daylight
golden_hour	4	warm golden-hour sunset light
twilight	5	twilight / evening light

Every instruction ends with an explicit preservation clause ("Keep the layout, furniture, objects and everything else exactly the same"), so that any structural change is unambiguously a failure rather than a matter of interpretation.

Settings. One seed (0), identical source files, identical prompts, same accelerator. The two models were run at **their own recommended sampling settings**, not matched ones: Qwen at 40 steps and CFG 4.0, FLUX at 28 steps and CFG 2.5. This is a deliberate system-level choice — it measures what each stack delivers as published — but it means the cost comparison bundles together three causes (parameter count, step count, per-step cost), which are separated numerically in the Results instead of being left implied.

Metrics.

- clip_gain — CLIP text-image similarity of the *edited* image to a target caption, minus the same value for the *unedited* source. A positive value means the edit moved the image toward the requested description. Measures task execution.

- clip_i — CLIP image-image cosine between source and output. Measures global retention.
- edge_ssim — SSIM computed on Sobel edge maps of source and output. Measures preservation of architectural geometry specifically, which is the fidelity constraint that matters for a property listing.
- latency_s, peak_vram_gb, and derived images_per_hour.

Baseline. An identity pass-through model is defined in run_w2.py as a no-op control, but **it was not executed**, so this pair has no non-learned floor. That is a gap relative to Pair 7, which does have one, and it is recorded as a limitation rather than glossed over.

Pipeline. Figure 1 shows the data flow as actually implemented. Note that the reporting stage reads *only* the results file — no figure in this report is assembled from the images directly, which is precisely what makes every table regenerable from results/w2.jsonl alone.

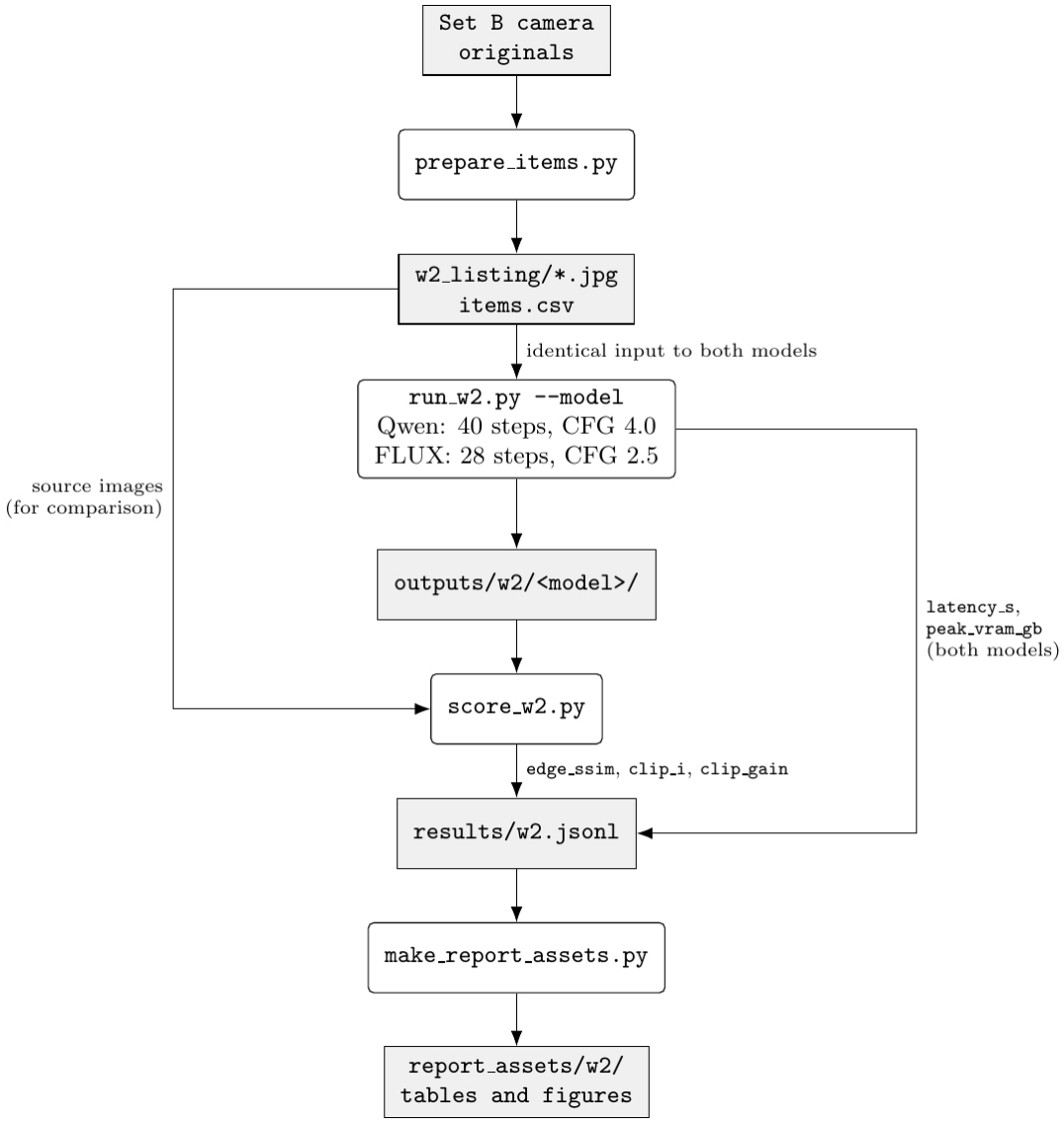


Figure 1. Pair 1 evaluation pipeline as implemented. run_w2.py is invoked once per model on byte-identical inputs, and each invocation writes both edited images and its own timing rows. score_w2.py reads the source images and the generated outputs and appends quality rows to the same results file. The reporting stage reads results/w2.jsonl only, never the images.

3.2.2 Results

Per-model means (n = 15, 95% bootstrap CI):

Model	clip_gain		clip_i		edge_ssim		latency (s)	peak VRAM (GB)
FLUX.1 Kontext [dev]	0.041	[0.025, 0.059]	0.886	[0.835, 0.930]	0.567	[0.501, 0.633]	17.52	33.78
Qwen-Image-Edit-2509	0.042	[0.024, 0.062]	0.904	[0.864, 0.938]	0.645	[0.571, 0.715]	50.94	58.00

Paired differences over items (Qwen minus FLUX, n = 15):

Metric	Mean diff	95% CI	Qwen higher on	Verdict
edge_ssim	+0.077	[0.027, 0.134]	11 / 15	**Qwen preserves structure better**
clip_i	+0.018	[-0.002, 0.039]	10 / 15	no clear difference
clip_gain	+0.001	[-0.005, 0.008]	6 / 15	**no clear difference**
latency_s	+33.42	[33.41, 33.44]	15 / 15	**FLUX far faster**
peak_vram_gb	+24.22	[24.22, 24.22]	15 / 15	**FLUX far lighter**

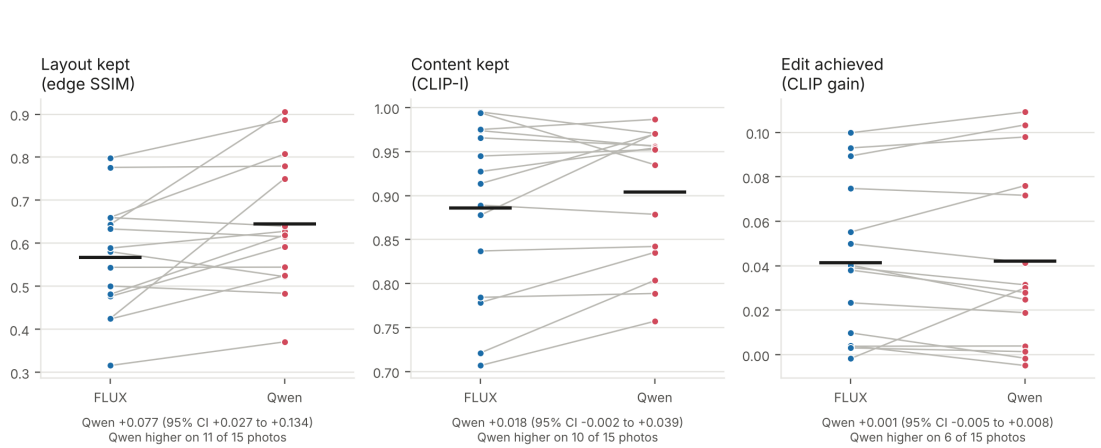


Figure 2. Paired per-item quality. Each point is one photograph; the two models are compared on the same item.

The models execute the instruction equally well. The clip_gain difference is +0.001 with an interval spanning zero, and Qwen is higher on only 6 of 15 photographs — as close to a tie as this design can produce. Whatever distinguishes these two models, it is not how well they carry out the requested edit.

Qwen preserves architectural structure better. The edge_ssim advantage of +0.077 excludes zero and holds on 11 of 15 photographs. Figure 3 shows this is not driven by a single edit family: Qwen leads on all four.

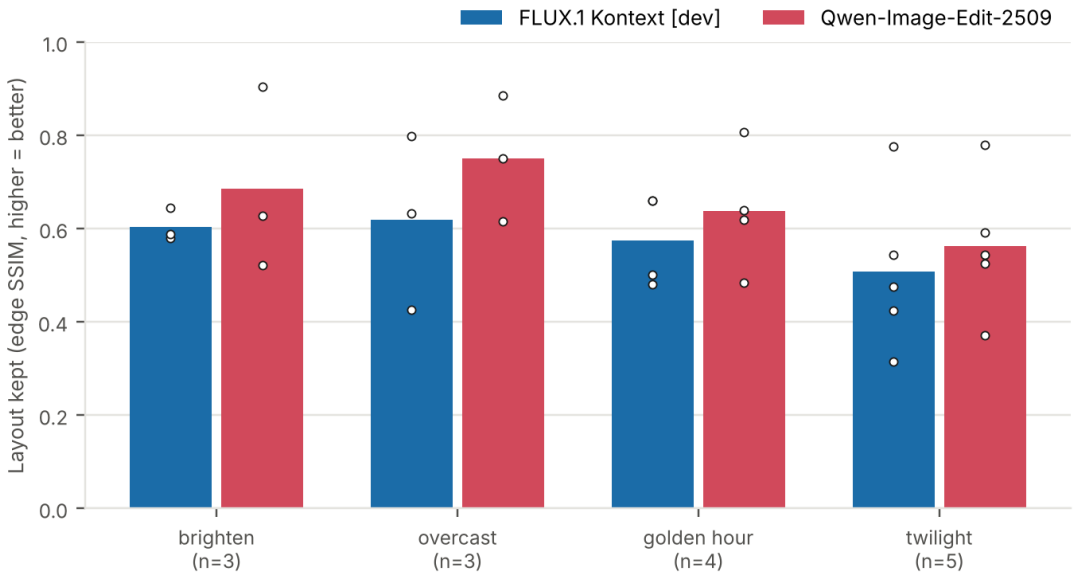


Figure 3. Structure preservation by edit family. Qwen leads on every family, so the aggregate is not an artefact of edit mix.

Edit difficulty varies enormously, and the aggregate conceals it. Broken out by family, clip_gain is near zero for brighten (0.006 FLUX, -0.002 Qwen) and large for twilight (0.082 FLUX, 0.092 Qwen):

Edit family	Photos	clip_gain (FLUX)	clip_gain (Qwen)	edge_ssim (FLUX)	edge_ssim (Qwen)
brighten	3	0.006	-0.002	0.604	0.685
overcast	3	0.008	0.018	0.619	0.751
golden_hour	4	0.042	0.031	0.576	0.638
twilight	5	0.082	0.092	0.507	0.562

A negative clip_gain on brighten for Qwen, and a near-zero one for FLUX, should not be read as "the models failed to brighten the room". It is more plausibly a statement about the metric: a modest exposure change barely moves CLIP's text-image similarity, so the measurement has almost no dynamic range on subtle edits. This is the same instrument weakness that dominates Pair 6, where the matched-versus-mismatched gap for CLIP text similarity is only 0.036. **clip_gain is informative for aggressive edits such as twilight and close to uninformative for subtle ones**, and the 15-item aggregate averages across both regimes.

The clip_i column also behaves as expected and confirms the metric is tracking something real: it falls monotonically as the requested edit becomes more aggressive, from 0.97 on brighten down to 0.77–0.81 on twilight.

Cost, decomposed. FLUX processes 205.4 photographs per hour against Qwen's 70.7 — a 2.9x throughput difference on identical hardware. That headline figure has three separable causes, and attributing it wholly to architecture would be wrong:

Component	Ratio	Source
Sampling steps	1.43x	40 steps (Qwen) vs 28 (FLUX), each model's own recommended setting
Per-step cost	2.03x	1.274 s/step (Qwen) vs 0.626 s/step (FLUX) — parameter count and architecture
Total latency	**2.91x**	50.94 s vs 17.52 s per photograph

About half of the throughput advantage comes from FLUX needing fewer sampling steps, which is a genuine property of its guidance distillation and counts in its favour. The remaining factor of two is per-step

cost, where a 12 B model is being compared with a 20 B one — a size difference, not evidence that one architecture is more efficient at equal capacity.

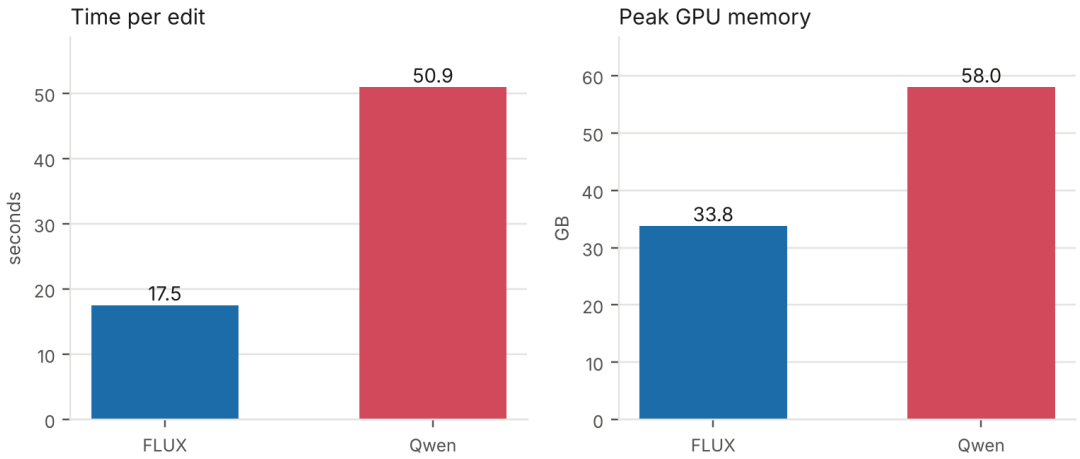


Figure 4. Throughput and peak memory. The 58 GB figure places Qwen beyond a single 40 GB accelerator.

The memory result is the harder operational constraint: Qwen's 58.0 GB peak exceeds a single A100-40GB, whereas FLUX's 33.8 GB fits with headroom. At the volume assumed in the scenario, this is the difference between one accelerator class and another.

3.2.3 Reflection and recommendation

Recommendation. Deploy Qwen-Image-Edit-2509 for listing photo normalisation, and be clear that the decision is made by licence rather than by quality. On the task metric the models tie — clip_gain differs by +0.001 with an interval spanning zero, and Qwen is higher on only 6 of 15 photographs — so editing quality cannot separate them. The axes that do separate them pull in different directions: Qwen preserves architectural structure better (edge_ssim +0.077, holding on 11 of 15 photographs), while FLUX processes 205.4 photographs per hour against Qwen's 70.7 and peaks 24 GB lower in memory (33.8 GB against 58.0 GB). Under the scenario's volume premise that would be a genuine cost-versus-fidelity dilemma, but FLUX.1 Kontext [dev] is non-commercial and Qwen is Apache 2.0, so the dilemma never has to be resolved. The recommendation reverses only if a commercial agreement with Black Forest Labs is obtained; at that point FLUX's 2.9x throughput makes it the cheaper default, and Qwen's structure preservation argues for reserving it for structure-sensitive edits.

Gaps and open issues. Three qualify the result. The identity baseline defined in run_w2.py was never executed, so this pair has no non-learned floor and nothing shows how much of the measured quality required a learned model at all. clip_gain has almost no dynamic range on subtle edits — 0.006 on brighten against 0.082 on twilight — so the metric can certify aggressive edits but not the gentle relighting that is this capability's most common job. And the comparison is system-level — 28 steps against 40, a 12 B model against a 20 B one — so nothing here supports a claim about the editing mechanisms themselves.

Future work. Run the identity baseline; it already exists in code. Beyond that, a tiered deployment is worth evaluating: route the volume through the faster model by default, and escalate to the structure-preserving one when an output check — edge_ssim against the source falling below a threshold set from this chapter's per-item spread — flags an unrequested structural change. What that threshold should be, and whether escalation pays for itself at the assumed volume, is an empirical question this evaluation does not answer.

4 Pair 2 — Identity-preserving portrait generation

InstantID vs PuLID v1.1

4.1 Selection

4.1.1 AI modelling techniques

Both models generate a new photograph of a **specific person** from a single reference image, with no per-subject fine-tuning. For the marketplace, the use case is agent profile and marketing imagery: one reference photograph of an agent, re-rendered as a studio headshot, a shot outside a listed property, or an open-home scene, without booking a photographer for each variant.

Both are **training-free identity injection** methods: a face-recognition embedding of the reference is injected into a frozen diffusion backbone so that the generated face matches the reference while the text prompt controls pose, attire, lighting and setting. Neither performs any gradient update at inference, which is what makes them viable at marketplace scale — the alternative lineage (per-subject fine-tuning) costs minutes to hours per agent.

They differ in how the identity signal is injected and balanced against the prompt and — critically for this evaluation — in the **base model each is designed to run on**:

	InstantID	PuLID v1.1
Origin	InstantX	ByteDance, NeurIPS 2024
Method	Identity embedding injected via an IP-Adapter-style cross-attention path, combined with a ControlNet-style spatial control on facial keypoints	Contrastive alignment of the identity branch against the base model's own behaviour
Base model used	stabilityai/stable-diffusion-xl-base-1.0	Lykon/dreamshaper-xl-lightning
Sampling	30 steps, CFG 5.0, control scale 0.8	4 steps, CFG 1.2, id scale 1.0
Repository licence	Apache 2.0	Apache 2.0

The base models differ, and this is the dominant confound in the pair. PuLID runs on a Lightning-distilled SDXL finetune at 4 steps; InstantID runs on vanilla SDXL at 30. Both are each method's *own recommended configuration* — the Lightning base is the one named in PuLID's v1.1 demo script — so this is a legitimate system-level comparison of two published stacks. It is not a controlled comparison of two identity-injection mechanisms, and no claim of that kind is made below. Differences in speed, aesthetic quality and prompt adherence may stem from the base checkpoint rather than from the identity method.

Week 3's material covers this pair's lineage directly: textual inversion, DreamBooth and LoRA fine-tuning, and then training-free feature injection. InstantID and PuLID are the endpoint of that progression — they supersede per-subject fine-tuning, which costs minutes to hours of training per agent, with a single forward pass, and that cost collapse is what makes the marketplace use case viable at all. [Confirm against the Week 3 deck.]

4.1.2 Modelling context

Item	InstantID	PuLID v1.1
Objectives	Produce usable agent portraits from one reference, consistently enough to publish without review	Same, with far lower inference cost per image
Tasks	Headshot, exterior, interior and open-home portrait scenarios from a single reference face	Identical task set

Functions	Single-pass generation; keypoint control additionally fixes face pose and position	Single-pass generation; no spatial control
Framework	Frozen SDXL + identity cross-attention adapter + facial-keypoint ControlNet	Frozen Lightning SDXL + contrastively aligned identity branch
Algorithms	Face embedding from a face-recognition encoder; adapter and control signal applied at each denoising step	Contrastive alignment objective trained to insert identity while disturbing the base distribution as little as possible
Governance	Processes **biometric data** (face embeddings) and produces photorealistic likenesses. Code is Apache 2.0, but the **InsightFace face models it depends on are licensed for non-commercial research only**	Same biometric and likeness exposure; also Apache 2.0, and also dependent on an InsightFace face encoder in the pipeline as implemented here
Advantages	Markedly more reliable (quantified below); spatial control gives predictable framing	3.4x faster end-to-end; better prompt adherence; smaller memory footprint

Three governance points carry real weight for this pair, and they differ in kind from those in Pair 1.

Biometric processing. Unlike photo editing, this capability requires the marketplace to collect and process agents' facial biometrics. That engages consent, retention and purpose-limitation obligations that arise nowhere else in this report, and it is a category of personal information treated as sensitive in most privacy regimes.

Likeness and misuse. A system that renders a convincing photograph of a named, identifiable person from one reference is equally a system that can place that person in a scene they were never in. For an agent profile this is benign; the same capability with a different prompt is not. Any deployment needs the agent's explicit consent for each generated use, not a blanket consent given at onboarding.

A dependency-level licensing constraint affecting both models. InstantID's own disclaimer states the code is available for academic and commercial use, but also that "both manual-downloading and auto-downloading face models from insightface are for non-commercial research purposes only." Because both pipelines as implemented here rely on an InsightFace face encoder, **this restriction applies to both models and therefore does not decide the pair — it blocks the capability as built**. A commercial deployment would need the face encoder swapped for a permissively licensed alternative, and the evaluation re-run, because changing the encoder changes the identity signal itself. This is a more serious finding than Pair 1's licence issue, which at least had a compliant alternative within the pair.

4.2 Evaluation

4.2.1 Methodology

Items. 4 reference identities x 4 scenarios = 16 items, each run on 3 fixed seeds (0, 1, 2) = **48 generations per model**. The scenarios are headshot, exterior, interior and open_home, with prompts written for the marketplace context (for example, a real-estate agent in a navy blazer under soft studio lighting).

Reference faces, and why they are mostly synthetic. One identity (me) is the author's own face; the other three (syn_a, syn_b, syn_c) are **synthetic faces generated for this assignment** by make_synthetic_refs.py. This was a deliberate ethical choice: evaluating an identity-preservation system means repeatedly generating photorealistic images of specific people, and doing that with third parties' photographs would have meant processing their biometric data without a research consent process. Using the author's own likeness plus synthetic references avoids that entirely.

The cost of that choice is stated plainly: **synthetic reference faces may not behave like real ones under face-recognition encoders**, since they come from the same generative family as the systems under test. The

single real identity (me) is the only check against this, and at n = 1 it is a weak one. Notably, me scores lowest of the four identities for both models (0.818 InstantID, 0.799 PuLID), which is consistent with synthetic faces being easier to reproduce — but with only one real identity this is a hint, not a result.

Settings. Each model ran at its own recommended configuration (see Selection). Identical prompts, identical reference crops, identical seeds, same accelerator.

Metrics.

- id_facenet — **primary** identity metric: cosine between FaceNet (VGGFace2) embeddings of the reference and generated faces.
- id_arcface — **secondary** identity metric, from an independent backbone (InsightFace buffalo_l). Two backbones are used because a single recogniser can be tracked by an artefact rather than by identity — which is exactly what happened.
- clip_t — prompt adherence.
- face_found — whether any face was detected in the output (a validity gate).
- face_fraction — the share of the frame occupied by the face box, recorded specifically to test whether identity scores are confounded by how large the model renders the face.
- id_facenet_heldout — identity measured against *different* photographs of the same person, available only for me.
- latency_s, peak_vram_gb.

Baseline. As in Pair 1, an identity pass-through is defined in run_w3.py but **was not executed**, so there is no non-learned floor for this pair.

4.2.2 Results

Per-model means (n = 48, 95% CI):

Metric	InstantID	PuLID v1.1
id_facenet (primary)	0.854 [0.842, 0.868]	0.834 [0.790, 0.864]
id_arcface (secondary)	0.769 [0.759, 0.779]	0.650 [0.626, 0.674]
clip_t	0.270 [0.261, 0.278]	0.306 [0.299, 0.313]
face_found	1.000	1.000
face_fraction	0.172 [0.163, 0.182]	0.058 [0.048, 0.069]
latency_s	3.03	0.88
peak_vram_gb	13.71	12.00

Paired differences (PuLID minus InstantID, n = 48):

Metric	Diff	95% CI	Significant
clip_t	+0.037	[0.026, 0.046]	**yes — PuLID follows the prompt better**
id_arcface	-0.119	[-0.145, -0.094]	**yes — InstantID higher**
id_facenet	-0.020	[-0.065, 0.010]	no
face_fraction	-0.114	[-0.127, -0.100]	**yes — InstantID renders much larger faces**
latency_s	-2.147	[-2.204, -2.036]	**yes — PuLID 3.4x faster end-to-end**
peak_vram_gb	-1.710	[-1.715, -1.706]	**yes — PuLID lighter**

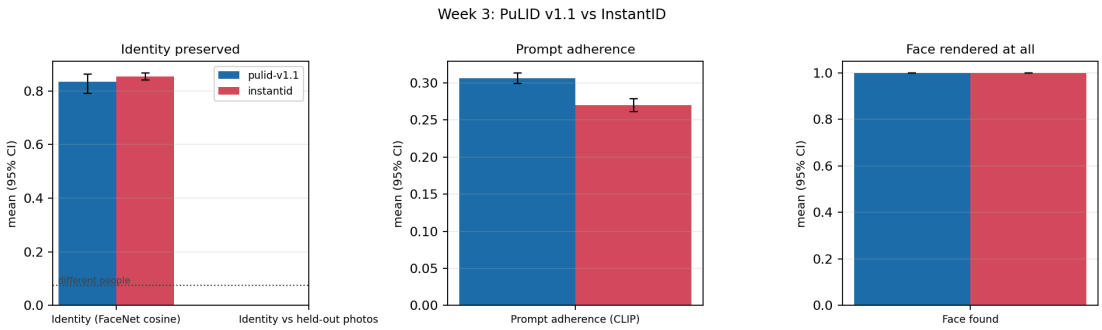


Figure 5. Headline identity and prompt-adherence results with 95% intervals.

The two identity backbones disagree, and the disagreement is systematic. On the secondary metric (ArcFace), InstantID leads by a large and significant margin (+0.119). On the **primary** metric (FaceNet), the difference is -0.020 and not significant. A report using only ArcFace would have declared a confident identity win that the primary metric does not support.

The face-size confound, quantified. InstantID renders faces at 17.2% of the frame against PuLID's 5.8% — a three-fold difference that is highly significant. Correlating face size with each identity metric across all 96 generations:

Correlation with face_fraction	id_arcface	id_facenet
Pooled across both models (n = 96)	**+0.741**	+0.124
Within InstantID (n = 48)	+0.116	-0.376
Within PuLID (n = 48)	+0.592	+0.231

The metric that favours InstantID is the metric that tracks face size (pooled r = +0.741), while the primary metric does not (+0.124). That is a coherent explanation for the backbone disagreement: a recogniser given three times as many facial pixels yields a higher embedding similarity largely irrespective of identity fidelity.

This evidence is suggestive, not conclusive, and the report does not overstate it. With only two models, a pooled correlation mixes the between-model difference with any genuine within-model effect, and the within-model correlations are weaker and inconsistent in sign (-0.376 for FaceNet within InstantID). The controlled test — normalising the face crop to a fixed size before computing embeddings, then re-scoring — was not run, and it is the required follow-up before the confound can be called established.

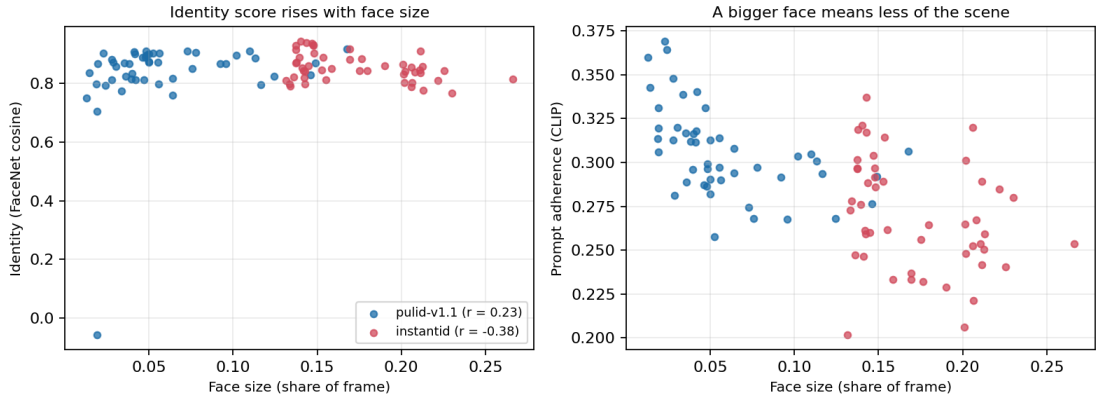


Figure 6. Identity score against face size, per model. The metric that separates the models is the one that tracks face area.

Reliability separates the models more cleanly than the mean does. Breaking the primary metric down by reference identity and by scenario shows InstantID is uniformly consistent while PuLID is not:

Breakdown	InstantID	PuLID v1.1

by identity: me	0.818 [0.800, 0.835]	0.799 [0.776, 0.819]
by identity: syn_a	0.908 [0.886, 0.926]	0.890 [0.876, 0.902]
by identity: syn_b	0.863 [0.849, 0.880]	0.865 [0.847, 0.881]
by identity: syn_c	0.829 [0.815, 0.843]	0.785 **[0.622, 0.880]**
by scenario: open_home	0.865 [0.842, 0.888]	0.748 **[0.589, 0.844]**

The two wide intervals trace back to **a single catastrophic failure**: on syn_c_open_home at seed 1, PuLID scored **-0.058** — a face was detected (face_found = 1.0) but it bears no resemblance whatsoever to the reference. InstantID scored 0.861 on the identical item. Across all 48 generations:

	InstantID	PuLID v1.1
Minimum id_facenet	0.767	**-0.058**
Median	0.845	0.868
Maximum	0.944	0.918
Standard deviation	**0.046**	0.139
Generations below 0.60	0 / 48	1 / 48

PuLID's *median* is higher than InstantID's. Its *variance* is three times greater, and it produced one output that is, for practical purposes, a photograph of a different person. For the stated use case — a portrait published on an agent's public profile — those two facts are not equally important. A model that is usually marginally better but occasionally renders the wrong person forces a human review step onto every image, which erases most of the cost advantage that made it attractive.

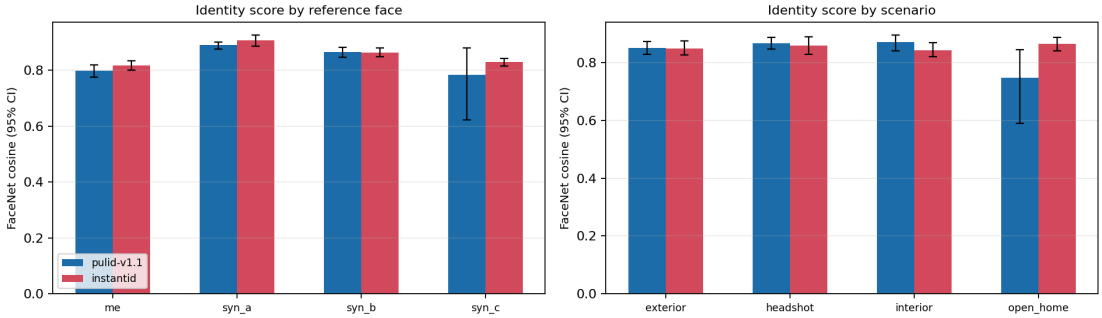


Figure 7. Identity score by reference face and by scenario. PuLID's wide intervals come from one failure, not from uniformly weaker performance.

Cost, decomposed. PuLID is 3.44x faster end-to-end (0.879 s vs 3.026 s). As in Pair 1, the headline conceals the mechanism:

Component	Ratio	Direction
Sampling steps	7.5x	30 (InstantID) vs 4 (PuLID)
Per-step cost	2.18x	0.101 s/step (InstantID) vs 0.220 s/step (PuLID) — **InstantID is faster per step**
Total latency	**3.44x**	in PuLID's favour

PuLID's entire speed advantage comes from the Lightning base model needing 4 denoising steps instead of 30. Per step it is **more than twice as expensive** as InstantID — the opposite of the impression the headline gives. The speed result is therefore a property of the base checkpoint, not of contrastive alignment, and it would not survive running both methods on the same backbone.

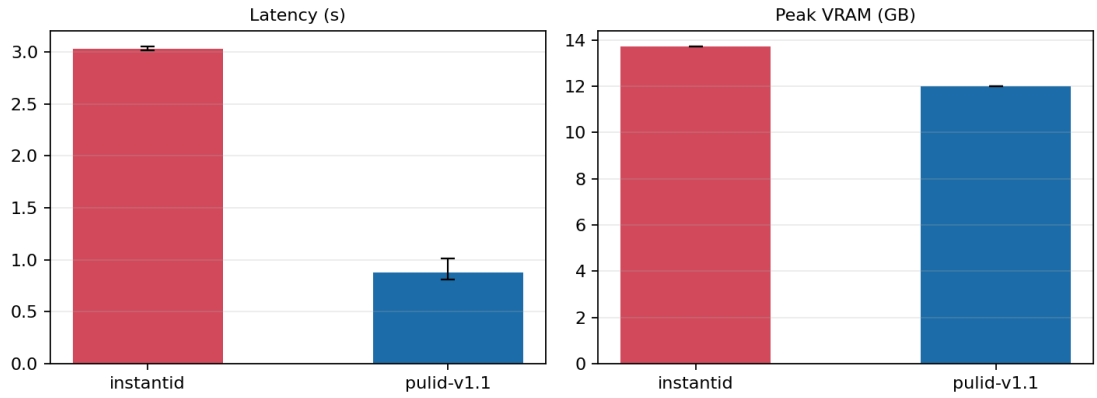


Figure 8. Latency and peak memory per generated portrait.

A failed control. id_facenet_heldout — identity measured against *different* photographs of the same person, the strongest available test of genuine identity preservation — returned nan for all 12 applicable items in both models (0 valid of 12). The control did not execute and provides no evidence. It is reported rather than omitted, and it is the single largest gap in this pair: without it, every identity number here measures similarity to **one specific reference photograph** rather than to the person.

4.2.3 Reflection and recommendation

Recommendation. As built, neither pipeline is deployable: both route the identity signal through an InsightFace face encoder whose models are restricted to non-commercial research, so the capability is blocked at the dependency level regardless of what the metrics say. If the encoder is replaced with a permissively licensed alternative — which changes the identity signal itself and therefore invalidates every number in this chapter, requiring a re-run — that re-run should start from InstantID, on reliability rather than on mean quality. On the primary metric the models do not separate (id_facenet −0.020, interval spanning zero), and InstantID's large ArcFace advantage tracks how large it renders the face (pooled r = +0.741, against +0.124 for FaceNet), so it is not evidence of better identity fidelity. What separates the models is the failure distribution: standard deviations of 0.046 against 0.139, and one PuLID generation in 48 — syn_c_open_home at seed 1, scoring −0.058 — that is, for practical purposes, a photograph of a different person. For a portrait published on an agent's public profile, a slightly lower mean is tolerable and an occasional wrong face is not, because the wrong face forces human review onto every output and erases the cost advantage that made generation attractive.

Gaps and open issues. The held-out identity control returned nan for all 12 applicable items, so nothing in this pair distinguishes "resembles the reference photograph" from "resembles the person" — the largest gap in the chapter. The face-size confound is quantified but not controlled; the fixed-crop re-scoring proposed in Results is the required follow-up before it can be called established. And three of the four identities are synthetic, with the single real identity scoring lowest for both models (0.818 InstantID, 0.799 PuLID) — a hint that synthetic references are easier to reproduce, not a result.

Future work. In order: repair the held-out control, whose failure was a file-linking defect rather than a design one; re-score both models on face crops normalised to a fixed size, settling the confound; add further real identities under an explicit consent process; and only then evaluate a replacement face encoder. The governance work — per-use consent, retention limits and misuse review for a named person's likeness — is a precondition for deployment, not a follow-up to it.

5 Pair 3 — Walkthrough reconstruction from a photo set

Nerfacto (NeRF) vs Splatfacto (3D Gaussian Splatting)

5.1 Selection

5.1.1 AI modelling techniques

Both models solve **novel-view synthesis**: given a set of photographs of a room with known camera poses, build a representation from which the scene can be rendered from viewpoints that were never photographed. For the marketplace this is the walkthrough capability — an agent walks through a property with a phone, and buyers can afterwards move freely through the space instead of clicking between fixed stills.

The two models embody the field's central architectural shift:

- **Nerfacto** is a **neural radiance field**. The scene is stored *implicitly* in the weights of a small network queried along camera rays, and an image is produced by volumetric ray marching — sampling many points per pixel and integrating colour and density.
- **Splatfacto** is **3D Gaussian splatting**. The scene is stored *explicitly* as a collection of anisotropic 3D Gaussians with position, covariance, opacity and colour, and an image is produced by projecting and rasterising them — the same primitive operation a GPU performs for conventional graphics.

The implicit-to-explicit shift is why the rendering results below differ by three orders of magnitude: ray marching requires many network evaluations per pixel, whereas rasterising Gaussians is a single pass over primitives on hardware built for exactly that.

Both are the Nerfstudio reference implementations, which is what makes this pair a fair comparison. They share a data loader, pose estimation, training harness and evaluation code; only the scene representation differs. This is the most tightly controlled pair in the report — Pairs 1 and 2 each compare two independently packaged stacks, whereas here a single framework holds everything constant except the representation itself.

This pair is Week 4's lecture made concrete: the deck's arc runs from the representation zoo and volume rendering through NeRF to 3D Gaussian Splatting, and Nerfacto and Splatfacto are the reference implementations of its two endpoints. The 846-fold rendering gap measured below is the implicit-to-explicit shift the lecture describes — rasterising explicit Gaussians supersedes ray-marching an implicit field, and it is what turns novel-view synthesis from a rendering-farm problem into an interactive product. [Confirm against the Week 4 deck.]

5.1.2 Modelling context

Item	Nerfacto	Splatfacto
Objectives	Reconstruct a room from an agent's phone capture at acceptable quality	Same, with interactive playback as a hard requirement
Tasks	Novel-view synthesis of interior scenes from ~100 casually captured frames	Identical
Functions	Train once per property, then render arbitrary viewpoints on request	Same, plus real-time rendering in a browser-class viewer
Framework	Nerfstudio 1.1.5 + nerfacc 0.5.2; MLP with hash-grid encoding; volumetric ray marching at 32,768 rays per chunk	Nerfstudio 1.1.5 + gsplat 1.4.0; explicit Gaussian primitives; tile-based differentiable rasterisation
Algorithms	Radiance-field regression with ray-sampling; 30,000 iterations	Gaussian optimisation with adaptive densification and pruning; 30,000 iterations
Governance	Capture of a real dwelling: occupants'	Identical exposure; additionally the explicit

	possessions, documents and neighbouring properties are reconstructed incidentally	point cloud is trivially inspectable and editable, which cuts both ways
Advantages	Mature, handles view-dependent effects smoothly	Dominant on every measured axis here (below); permissively licensed

Licensing is clean for this pair, in a way that is instructive after Pair 2. Nerfstudio is Apache 2.0, nerfacc is MIT, and **gsplat — the rasterisation backend Splatfacto uses — is released under the Apache License 2.0**, confirmed from the gsplat paper, which describes it as a standalone library that reimplements the original 3D Gaussian Splatting work rather than deriving from it. The original Kerbl et al. reference implementation carries its own separate terms, which were **not verified for this report**; because Splatfacto depends on gsplat and not on that codebase, those terms do not bind this deployment.

This is the mirror image of Pair 2. There, a permissively licensed method was blocked by a non-commercial *dependency*. Here, a permissively licensed *reimplementation* of a restrictively licensed research codebase removes the obstacle entirely. **Licence provenance has to be traced through the dependency tree, not read off the top-level repository**, and it decided deployability in both pairs.

The governance row deserves emphasis for a different reason than elsewhere. Reconstructing a lived-in dwelling captures whatever happens to be in it. Both methods rebuild documents on a desk, items visible through a window and the neighbouring property — and the capture consent obtained from a vendor does not obviously extend to a tenant's belongings or a neighbour's garden.

5.2 Evaluation

5.2.1 Methodology

Scene and capture. One room captured with a phone: 100 input frames at 1440 px maximum side. Camera poses were estimated **once** with COLMAP (pycolmap 4.2.1, exhaustive matching), and the **same poses were used by both models**, so pose error is held constant and cannot explain any difference. A quality gate required at least 90% of frames to be registered; **99 of 100 were placed (99%)**, so the gate passed. Five further images were dropped before processing for having a different resolution.

Split. Every 8th frame is held out and never trained on, giving **13 evaluation views**. Both models are evaluated on exactly these views.

Settings. 720x540 (downscale factor 2) for training and evaluation, 30,000 iterations — each model's own default budget, identical between them — and **3 training seeds per model**. The unit of analysis is the held-out photograph, with each photograph's score averaged over seeds.

Metrics. psnr, ssim, lpips (perceptual, lower is better) on held-out views; plus train_time_s, render_fps, peak_vram_gb, model_size_mb, and n_gaussians for Splatfacto.

Hardware and versions. Runs were executed on an NVIDIA RTX PRO 6000 Blackwell (97,887 MiB), with the full toolchain recorded in results/w4_summary.json: Python 3.12.13, torch 2.11.0+cu128, nerfstudio 1.1.5, gsplat 1.4.0, nerfacc 0.5.2, lpips 0.1.4, COLMAP 3.7-2. Note that this is **not** the Colab accelerator used elsewhere in this report, so absolute timings are not comparable across pairs; the within-pair ratios are.

Stated limitations of the design, recorded in config.yaml before the run rather than discovered afterwards:

1. Held-out frames sit *between* training frames of the same walk-through, so this measures **interpolation, not extrapolation** to genuinely novel viewpoints. A buyer who moves somewhere the agent did not walk falls outside what this evaluation tests.
2. train_time_s is wall-clock for the whole ns-train process, including start-up and image caching, not pure optimisation time.
3. render_fps is measured at 720x540, and Nerfacto's figure depends on its ray chunk size.

5.2.2 Results

Held-out view quality (n = 13 views, seeds averaged, 95% CI):

Metric	Nerfacto	Splatfacto
psnr (dB)	19.76 [18.07, 21.29]	**22.19 [20.51, 23.80]**
ssim	0.707 [0.643, 0.763]	**0.768 [0.716, 0.815]**
lpips (lower better)	0.247 [0.198, 0.303]	**0.198 [0.156, 0.245]**

Paired differences (Splatfacto minus Nerfacto, n = 13 views):

Metric	Diff	95% CI	Significant
psnr	+2.424 dB	[0.766, 3.849]	**yes**
ssim	+0.061	[0.036, 0.087]	**yes**
lpips	-0.049	[-0.075, -0.026]	**yes**

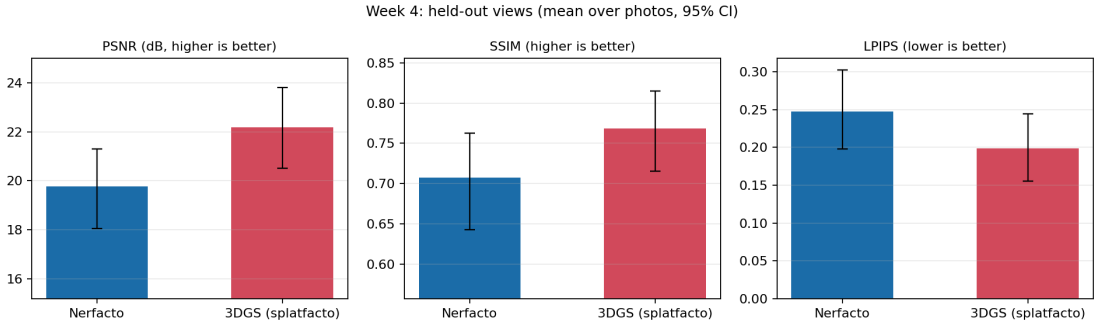


Figure 9. Held-out view quality. All three metrics favour Splatfacto with intervals excluding zero.

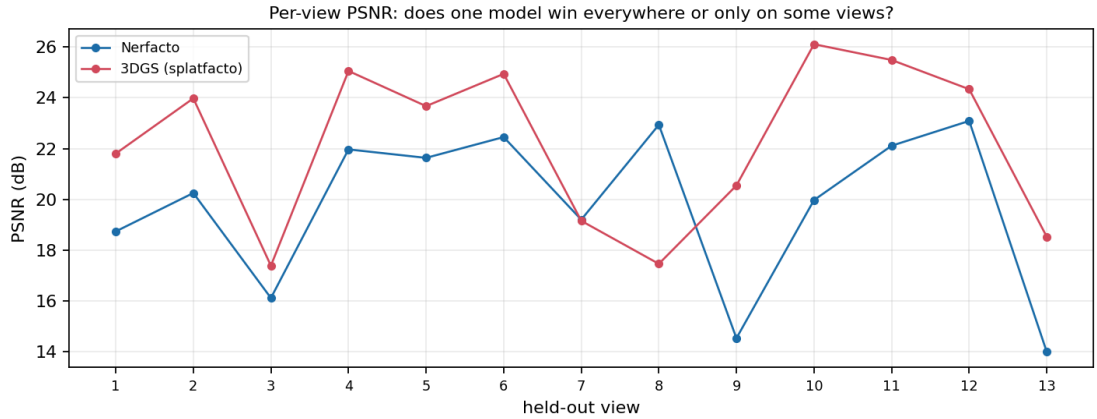


Figure 10. Per-view PSNR. Splatfacto leads on the great majority of individual held-out photographs, so the aggregate is not driven by a few easy views.

Run-level cost (3 seeds each):

Metric	Nerfacto	Splatfacto	Ratio
train_time_s	1544.6 (25.7 min)	**180.0 (3.0 min)**	**8.6x**
render_fps	0.516	**436.6**	**846x**
peak_vram_gb	4.076	**1.510**	2.7x
model_size_mb	231.6	221.6	1.05x

n_gaussians	—	314,259	—

Seed stability (mean held-out PSNR across 3 training seeds):

Model	Mean PSNR	SD across seeds
Nerfacto	19.763	0.520
Splatfacto	22.188	**0.262**

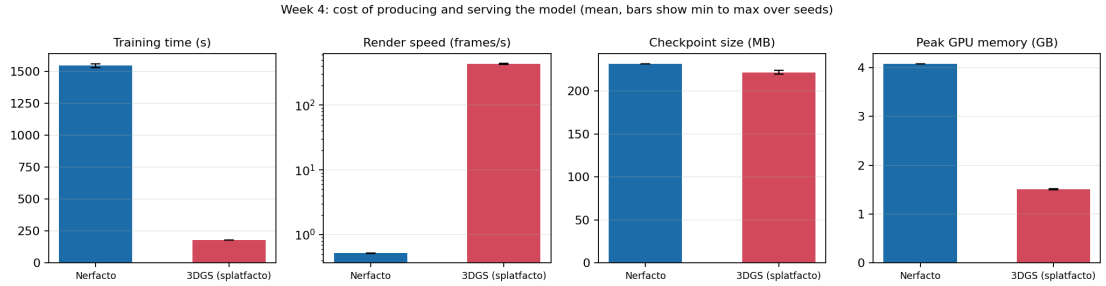


Figure 11. Training time and rendering throughput.

This is the only pair in the report where one model wins on every axis that was measured. Splatfacto is better on all three quality metrics with all three intervals excluding zero, trains 8.6x faster, uses 2.7x less memory and is twice as stable across training seeds. There is no trade-off to argue, which is itself unusual — every other pair in this report required one dimension to be weighed against another.

The rendering result is the one that decides deployability, and it is not close. Nerfacto renders at **0.516 frames per second** — roughly two seconds per frame. Splatfacto renders at **436.6 fps**, an **846-fold** difference. This is not a matter of degree. At half a frame per second an interactive walkthrough is impossible; the capability would have to be cut back to pre-rendered video along a fixed path, which is a different and much less valuable product. Splatfacto's figure leaves headroom for far weaker client hardware than the server GPU used here. The explicit-versus-implicit representation choice, discussed in Selection, is the direct cause.

Model size is the one axis where neither model wins, and it is a real constraint. Both produce artefacts of roughly 225 MB (231.6 MB Nerfacto, 221.6 MB Splatfacto). Splatfacto's 3.0 minutes of training makes per-listing reconstruction economically viable at the volume assumed in the Introduction, but shipping 225 MB per property to a browser is not viable as it stands, and the evaluation did not measure what Gaussian pruning or compression would cost in quality. That is the obvious next experiment, and it is unaddressed here.

Cost in dollars could not be computed. aud_per_run is nan in w4_cost.csv because the --units-per-hr argument was not passed to the reporting script. The underlying GPU-minutes (25.7 vs 3.0 per run) are reported instead, and they are provider-independent.

5.2.3 Reflection and recommendation

Recommendation. Deploy Splatfacto. This is the only pair in the report with no trade-off to argue: Splatfacto wins all three quality metrics with intervals excluding zero, trains 8.6x faster, uses 2.7x less memory and is twice as stable across seeds. Even had quality tied, rendering would decide it alone — 0.516 frames per second cannot serve an interactive walkthrough, so Nerfacto would collapse the product into pre-rendered video on a fixed path, while 436.6 fps leaves headroom for client hardware far weaker than the server GPU used here. At 3.0 minutes of training per scene, per-listing reconstruction is viable at the assumed volume in a way that 25.7 minutes is not. The recommendation holds under one condition: the artefact problem below gets solved, because roughly 225 MB per property is not servable as it stands.

Gaps and open issues. One scene was reconstructed, so the result is a strong claim about this room and a weaker one about all rooms. The design measures interpolation between captured frames; a buyer who

moves somewhere the agent did not walk falls outside what was tested. The artefact size is unaddressed — Gaussian pruning and compression were never measured, so the quality cost of making the product servable is unknown — and no dollar cost could be computed, because the reporting script was not given a units-per-hour figure. The largest unmeasured quantity is not in the numbers at all: the capture burden on the agent, who must walk every room methodically with a phone. That, not training time, is the likely deployment blocker.

Future work. Measure compressed artefact size against held-out quality, so the servability question has numbers. Extend the evaluation to further rooms, including difficult ones — mirrors, windows, clutter. Add an extrapolation test with views genuinely outside the capture path. And instrument the capture itself — time on site, retake rate — before any rollout decision is made.

6 Pair 4 — Single photo to 3D asset

TRELLIS.2-4B vs Hunyuan3D 2.1

6.1 Selection

6.1.1 AI modelling techniques

Both models take **one photograph of an object** and produce a textured 3D mesh. For the marketplace, the use case is virtual staging: building a library of furniture and decor assets from ordinary seller photographs, to be placed into empty or re-styled rooms.

This is a fundamentally harder problem than Pair 3, and the difference is worth stating precisely. Pair 3 had roughly 100 photographs of one room, so the unseen geometry was *constrained* by parallax between views — the task was reconstruction. Here there is a single viewpoint and no parallax at all, so the back and sides of the object are **generated, not reconstructed**. Nothing in the input determines them. That makes the question "is the mesh correct?" unanswerable without ground-truth scans, which this evaluation does not have, and it is why the metrics below measure *agreement with the input photograph* rather than geometric accuracy.

Both models are generative 3D systems that produce a shape and then a texture, differing in the latent representation used for shape. Both are substantially larger and more recent than the Pair 3 models, and neither is a reconstruction method in the classical sense.

Week 5 covered generation rather than reconstruction: few-shot NeRF propped up by CLIP priors, DreamFields, and DreamFusion's score distillation sampling, which optimises a 3D representation per prompt through hours of gradient descent. TRELLIS.2 and Hunyuan3D 2.1 supersede that paradigm with feed-forward generation — one pass producing shape and texture — and the minutes-scale latencies below are the practical consequence. [Confirm against the Week 5 deck.]

6.1.2 Modelling context

Item	TRELLIS.2-4B	Hunyuan3D 2.1
Objectives	Build a reusable staging asset library from seller photographs at low cost per item	Same
Tasks	Single-image to textured mesh for furniture and decor	Identical task set
Functions	Shape generation followed by texture synthesis and export to GLB	Same, with a markedly different artefact profile
Framework	microsoft/TRELLIS.2-4B; pipeline additionally loads third-party components (background removal, feature extractors) and NVIDIA differentiable-rasterisation code	tencent/Hunyuan3D-2.1; two-stage shape then texture
Algorithms	Structured latent 3D generation	Shape diffusion followed by texture synthesis
Governance	Core model and code **MIT**, but the pipeline pulls gated third-party models — including **RMBG-2.0, which is CC BY-NC 4.0 and requires a commercial agreement with BRIA** — plus NVIDIA nvdiffrast/nvdiffrec terms	**Tencent Hunyuan Community Licence**: territory excludes the EU, UK and South Korea; a separate Tencent licence is required above **1 million monthly active users**; deployments must disclose that Tencent is not affiliated; outputs may not be used to improve other AI models

Advantages	Permissive core licence; much lower peak VRAM	Web-deliverable artefacts (quantified below); faster and far more predictable runtime
Also required	Disclosure that staged furniture is generated, not photographed (Premise 3)	Identical disclosure obligation

Licensing is again decisive, and for this pair it constrains both models in different ways.

Hunyuan3D 2.1 is governed by the Tencent Hunyuan Community Licence. Three clauses matter here. The **Territory** is defined as worldwide *excluding the European Union, the United Kingdom and South Korea*, and use outside the Territory is unlicensed — irrelevant for an Australian marketplace, but fatal to any European expansion. More seriously, where a licensee's products exceed **1 million monthly active users**, the licensee "is not authorized to exercise any of the rights under this Agreement" until Tencent grants a separate licence at its sole discretion. **The scenario's own volume premise makes this threshold likely to bind**: a national property marketplace plausibly exceeds a million monthly users, so the model that wins this pair on technical grounds requires a discretionary negotiation with Tencent before it can be deployed at the assumed scale. The licence also requires deployments serving third parties to disclose that Tencent is not affiliated, and it forbids using outputs to improve other AI models — which rules out using generated assets as training data.

TRELLIS.2-4B has an MIT core, which looks cleaner, but its pipeline loads gated third-party components. One of those, **RMBG-2.0, is released under CC BY-NC 4.0, and its maintainers state that "commercial use is subject to a commercial agreement with BRIA."** Whether RMBG is actually invoked when pre-masked RGBA inputs are supplied, as they were here, **was not verified**, and the NVIDIA rasterisation terms were likewise not examined. The honest position is that TRELLIS's MIT badge does not by itself establish commercial usability.

This is now the fourth consecutive pair in which **licensing lives in the dependency tree rather than the top-level repository, and in which it constrains deployment more sharply than any quality metric**. The pattern is taken up in the synthesis chapter.

6.2 Evaluation

6.2.1 Methodology

Items. 13 furnishing and decor objects photographed for this assignment (topiary, lamp, book, tumbler, chair, hand chair, daybed, pouf, armchair, coffee table, sofa, bin, heater), fixed before the run in set_c_manifest.csv. Each item is run on **3 seeds** (0, 1, 2), with a separate unrecorded warm-up call at seed 99 so that first-call compilation does not contaminate the latency figures.

Input control. Both models receive the **same background-removed RGBA PNG**, produced once with rembg (birefnet-general), with manual mask overrides where automatic removal failed. There is no per-model preprocessing.

Rendering control. Because the two models export meshes with very different topology, comparing them fairly requires rendering them identically. A **fixed 8-view renderer shared by both models** produces the images on which all similarity metrics are computed, so neither model is advantaged by its own export format.

Metrics. Since the unseen geometry is generated rather than reconstructed, agreement with the input photograph can only be judged from the input viewpoint and nearby views. Each similarity metric is therefore reported both as a best-view (_best) and a mean-over-8-views (_mean) figure:

- dino_best / dino_mean — DINOv2-base CLS cosine, rendered view against the source photo.
- clip_i_best / clip_i_mean — CLIP ViT-B/32 image-image cosine.
- sil_iou_best — silhouette intersection-over-union against the photo cut-out, which tests outline fidelity independently of texture.
- Artefact properties: n_faces, watertight, n_components, textured, glb_size_mb.

- Cost: latency_s split into shape and texture/export stages, peak_vram_gb, load_time_s.
- gen_ok as a success gate.

Explicitly not measured, recorded in config.yaml before the run: **geometric accuracy against ground truth** (no 3D scans of the objects exist) and **true PBR appearance under varying lighting**. Every quality statement below is therefore about *visual agreement with one photograph*, not about whether the object's true shape was recovered.

6.2.2 Results

Visual agreement with the input photo (n = 13 items, paired bootstrap):

Metric	TRELLIS2	Hunyuan3D	Diff (A-B)	95% CI	Verdict
dino_best	0.783	0.786	-0.003	[-0.056, 0.044]	no clear difference
dino_mean	0.660	0.654	+0.005	[-0.038, 0.046]	no clear difference
clip_i_best	0.922	0.923	-0.002	[-0.019, 0.015]	no clear difference
clip_i_mean	0.889	0.886	+0.004	[-0.014, 0.023]	no clear difference
sil_iou_best	0.805	0.807	-0.002	[-0.027, 0.025]	no clear difference

All five visual metrics return "no clear difference." Per-item win counts are nearly even across the board (7–6, 8–5, 6–7, 7–6, 5–8). On the question "does the mesh look like the photograph", these two models are indistinguishable at n = 13, and no amount of re-reading the means will change that.

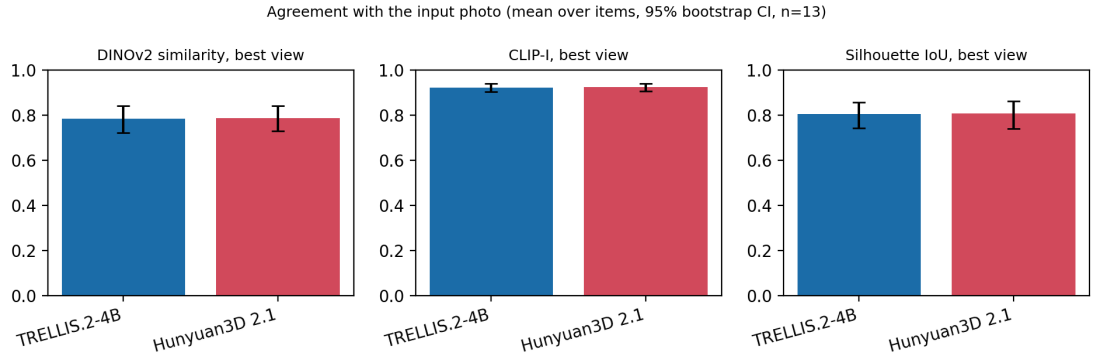


Figure 12. Visual agreement with the source photograph. All five metrics overlap.

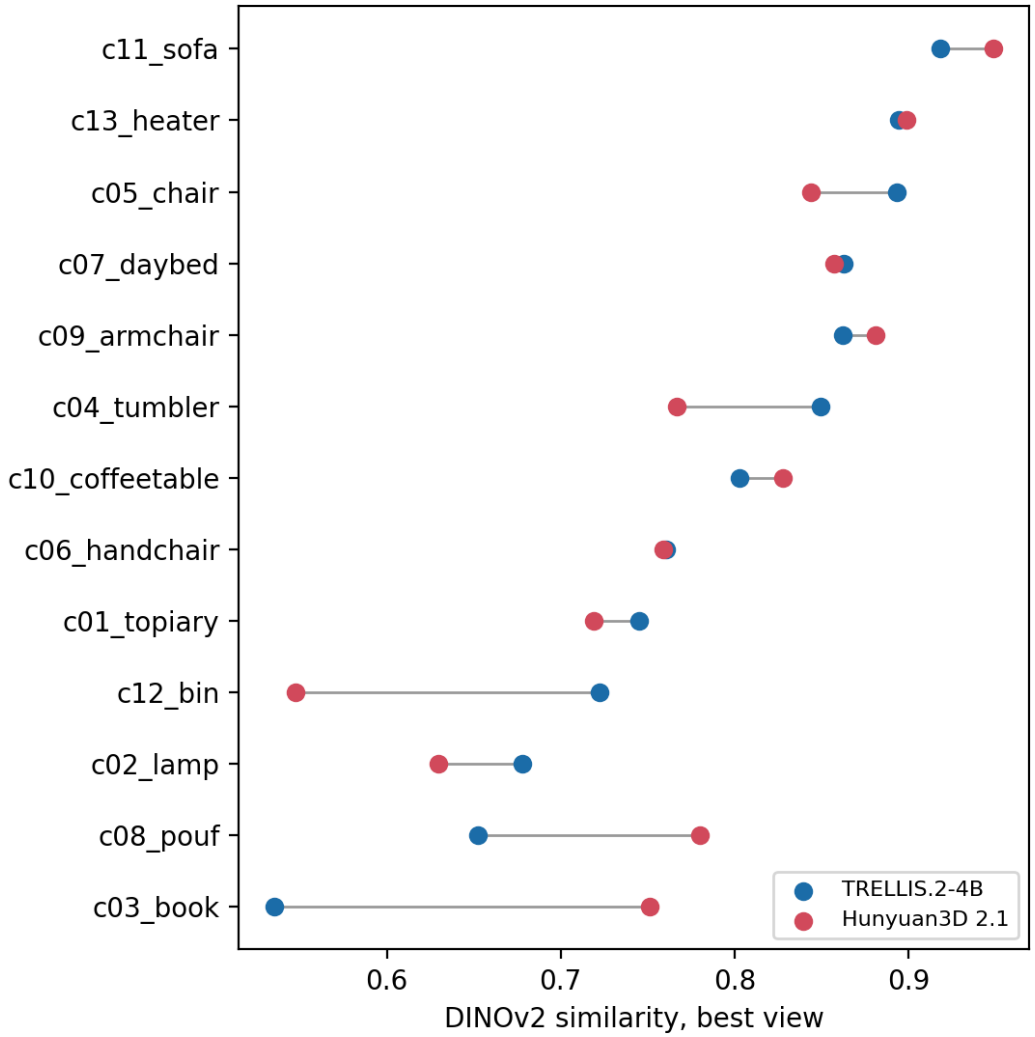


Figure 13. Per-item DINO agreement. Neither model leads consistently across the object set.

Metric validity control. The similarity metrics do discriminate, but unequally. Scoring each output against a *different* item's photograph:

Metric	Model	Matching photo	Different item's photo	Gap
dino_best	TRELLIS2	0.783	0.104	**0.679**
dino_best	Hunyuan3D	0.786	0.109	**0.676**
clip_i_best	TRELLIS2	0.922	0.732	0.189
clip_i_best	Hunyuan3D	0.923	0.738	0.186

DINO separates matched from mismatched content by about 0.68; CLIP image similarity by only 0.19. **DINO carries the discriminative weight and CLIP image similarity is the weaker instrument**, which is why the DINO columns are given primacy above. This matters beyond this pair: a CLIP-only evaluation would be operating with roughly a quarter of the usable range.

Artefact quality — where the models diverge completely (n = 13):

Property	TRELLIS2	Hunyuan3D	Ratio

n_faces	965,089	**40,000**	24.1x
watertight	**0.00 (0 / 13)**	**1.00 (13 / 13)**	—
n_components (mean)	96.3 (max 520)	**4.8** (max 49)	20.1x
glb_size_mb	69.45	**1.31**	**53.1x**
textured	1.00	1.00	—
generation_success_rate	1.00	1.00	—

Both models succeed on every item and both produce textured output, so neither fails outright. But Hunyuan3D returns a **watertight, 40,000-face, 1.3 MB mesh** — an asset that can be dropped into a web viewer as it is. TRELLIS2 returns a 965,000-face, 69 MB mesh that is watertight in **zero of 13 cases** and fragments into 96 connected components on average, with one item producing 520 separate pieces.

For the marketplace, a 53-fold file-size difference and a watertight-versus-not difference decide this pair outright, **even though the visual metrics tie**. A non-watertight mesh in ~96 disconnected pieces cannot be reliably relit, collision-tested or physically placed in a staged room without a repair and retopology stage that this evaluation did not measure.

Run cost (n = 13):

Metric	TRELLIS2	Hunyuan3D
latency_s (mean)	317.70 (167–912)	**116.10 (87–210)**
latency_shape_s	55.14	30.84
latency_texture_export_s	262.57	85.00
peak_vram_gb	**7.75** (2.85–15.35)	20.62
load_time_s	129.50	**82.74**

Hunyuan3D is 2.7x faster per item. TRELLIS2's texture and export stage accounts for 262 of its 318 seconds and shows extreme variance — **up to 912 seconds on a single item**, more than five times its own median. That is a throughput-planning problem as much as a cost one: batch scheduling against a stage whose duration varies five-fold is materially harder.

TRELLIS2 wins on exactly one axis: peak VRAM, at 7.75 GB against 20.62 GB, which would let it run on much cheaper hardware. Its VRAM also varies widely (2.85–15.35 GB), while Hunyuan3D's is essentially constant.

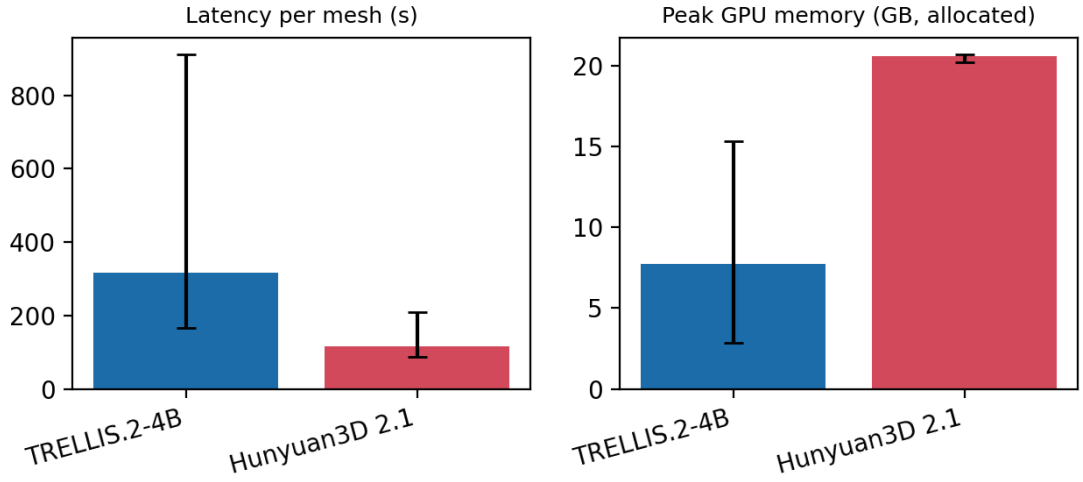


Figure 14. Latency and memory per generated mesh.

Seed replication produced no information. Within-item standard deviation across the three seeds is **exactly 0.000** for both models on dino_best and sil_iou_best. The pipelines are deterministic at these settings, so the three seeds are three identical runs. **Item-level variation is the only source of uncertainty in every interval reported above**, and the seed budget — three times the compute — bought nothing. This was discovered after the fact and stands as a methodological lesson rather than a design choice.

6.2.3 Reflection and recommendation

Recommendation. Choose Hunyuan3D 2.1 on artefact deployability, and be explicit that quality played no part in the choice: all five similarity metrics tie, so picking on visual agreement would be picking on noise. The deployability gap is total — watertight on 13 of 13 items against 0 of 13, 1.31 MB against 69.45 MB, 4.8 against 96.3 connected components — and Hunyuan3D is 2.7x faster with a predictable runtime, where TRELLIS.2's texture stage varies five-fold and hit 912 seconds on one item against a 318-second mean. A mesh that is never watertight and fragments into ~96 pieces cannot be relit, collision-tested or placed in a staged room without a repair and retopology stage this evaluation did not measure. TRELLIS.2 wins on exactly one axis, peak VRAM (7.75 GB against 20.62 GB), which matters for hardware cost and nothing else.

The condition on the recommendation is licensing, and it is severe. Hunyuan3D 2.1's community licence withdraws authorisation above 1 million monthly active users pending a separate licence at Tencent's discretion — and the scenario's own volume premise crosses that threshold. Three responses are open: negotiate with Tencent before further engineering, which is the direct route given that the scenario assumes the crossing scale; build a mesh-repair and retopology pipeline around TRELLIS.2 and re-evaluate, an unmeasured engineering cost on top of clearing TRELLIS.2's own CC BY-NC dependency; or judge the capability not yet viable at scenario scale and revisit when licensing or artefacts improve. Because the technical winner's licence binds precisely at the assumed volume, negotiation comes before engineering.

Gaps and future work. Geometric accuracy was never measured — no ground-truth scans of the objects exist — so every quality claim here is agreement with one photograph, not recovered shape; scanning even a few items would convert this pair from proxy to ground truth. PBR appearance under varying lighting is untested. The deterministic pipelines made the three-seed design three identical runs, so the seed budget bought nothing; future rounds should spend it on more items instead. And with CLIP image similarity showing only ~0.19 of usable range against DINO's ~0.68, DINO should carry future comparisons, with CLIP reported as a check.

7 Pair 5 — Listing photo to short motion clip

Wan2.2 TI2V-5B vs HunyuanVideo 1.5 (480p I2V)

7.1 Selection

7.1.1 AI modelling techniques

Both models perform **image-to-video generation**: given one listing photograph and a text prompt describing a camera movement, they synthesise a short clip that begins at the supplied photograph and animates it. For the marketplace, the use case is turning a static hero image into a few seconds of motion for listing previews and social placements, without sending a videographer.

Both encode the clip with a **causal 3D variational autoencoder** and denoise in that latent space with a diffusion transformer, conditioning jointly on the source image and the text. The temporal compression is what makes the task tractable: denoising 73 frames at pixel resolution would be prohibitive, whereas the latent sequence is a few times shorter in time and substantially smaller spatially.

They differ in how they trade sampling steps against per-step cost, which turns out to be the whole story of this pair's cost result:

	Wan2.2 TI2V-5B	HunyuanVideo 1.5 (480p I2V)
Developer	Alibaba (Wan)	Tencent (Hunyuan)
Configuration run	704x1280, 50 steps, CFG 5.0	480x848, 8 steps (step-distilled), CFG 1.0
Offload strategy	offload_model + text encoder on CPU	model_cpu_offload
Licence	Apache 2.0	Apache 2.0 (see below)

Licensing is clean here, and the contrast with Pair 4 is instructive. Wan2.2 is Apache 2.0. Two independent secondary sources report HunyuanVideo 1.5 as Apache 2.0 as well, one noting that the model's own licence table links to an Apache-2.0 LICENSE file; the official repository (tencent/HunyuanVideo-1.5) was **not reachable from the evaluation environment**, so this is reported from secondary sources rather than from the licence text itself.

That matters because it is **not** the licence Tencent applied to Hunyuan3D 2.1 in Pair 4, which carries territory exclusions and a 1 million MAU ceiling. **The same vendor uses different licences across its model families, so licence terms must be checked per model rather than inferred from the publisher.** Assuming Pair 4's restrictions applied here would have been wrong; assuming this pair's permissiveness applied there would have been worse.

Week 6 introduced video generation from its pioneering works — pixel-space video diffusion and the 3D and (2+1)D convolution machinery inherited from video recognition. Both models here are the current form of that lineage: latent-space diffusion transformers operating over a causal 3D VAE, which supersede pixel-space modelling by making the temporal sequence short enough to denoise at usable cost. [Confirm against the Week 6 deck.]

7.1.2 Modelling context

Item	Wan2.2 TI2V-5B	HunyuanVideo 1.5
Objectives	Animate a hero listing image at a cost per clip low enough to apply broadly	Same, with explicit emphasis on inference cost via step distillation
Tasks	Short camera-motion clips (push-in, pan, tilt) from one interior photograph	Identical task set
Functions	Single-pass generation of 73 frames at 24 fps from image	Same

	plus prompt	
Framework	Diffusion transformer over a causal 3D VAE latent; 5B parameters; run at 720p-class resolution with 50 denoising steps	Diffusion transformer over a causal 3D VAE latent; run at 480p with an 8-step distilled schedule
Algorithms	Standard multi-step denoising with classifier-free guidance at scale 5.0	Step-distilled sampling at guidance 1.0
Governance	Animating a still of a real property can imply spatial relationships the photograph did not show — a push-in through a doorway asserts what is beyond it. Disclosure obligations from Premise 3 apply	Identical exposure; the shorter clips and lower resolution reduce but do not remove it
Advantages	Higher output resolution; larger file artefacts suitable for primary placements	2.5x faster end-to-end; 23x smaller files; more actual motion

The governance row is specific to video and is worth separating from the image pairs. A still photograph makes a limited claim. A clip that moves the camera makes a **spatial** claim: that the room continues in a particular way, that a doorway leads somewhere, that a view exists from an angle that was never photographed. Because the unseen geometry is generated rather than observed, an animated listing can assert something about a property that nobody verified. This is a materially different disclosure problem from a relit photograph, and it maps directly onto the NSW bill's concern with images that mislead as to what a property actually contains.

7.2 Evaluation

7.2.1 Methodology

Items. 6 listing photographs (main_bed_wide, main_balcony_view, kitchen_b, bedroom_door, main_mirror_robe, main_fan_blind), each with a camera-motion prompt. One seed per item — a budget decision, stated rather than hidden. 73 frames at 24 fps (just over three seconds).

This is a system-level comparison, not a model-level one. The two stacks were run **as published**: Wan2.2 at 704x1280 with 50 steps and the text encoder offloaded to CPU, HunyuanVideo 1.5 at 480x848 with its 8-step distilled schedule. Resolution, step count, guidance scale and offload strategy all differ. The results below state what each published stack delivers to the marketplace; **they do not isolate architecture as the cause of any difference**, and no architectural claim is made.

Metrics. clip_t (prompt adherence); clip_i_src_mean, dino_src_mean, dino_src_min (per-frame similarity to the source photograph, mean and worst case); dino_consec and dino_first_last (temporal coherence and total drift); first_frame_ssim (does the clip actually start from the supplied photograph); motion_mad (mean absolute frame difference — how much motion there is at all); jitter_ratio (high-frequency temporal noise relative to total motion); luma_drift; plus latency_s, peak_vram_gb and file_mb. Scoring downsizes every frame to 224 px so that the resolution difference does not bias the embedding metrics.

Mismatch control. Each clip is additionally scored against a *different* item's source photograph.

7.2.2 Results

Quality, fidelity and motion (n = 6 items, paired bootstrap):

Metric	Wan2.2	HunyuanVideo 1.5	Diff (A-B)	95% CI	Verdict
clip_t	0.270	0.277	-0.007	[-0.020, 0.002]	no clear difference
clip_i_src_mean	0.960	0.913	+0.047	[0.006, 0.092]	**Wan higher**
dino_src_mean	0.962	0.939	+0.023	[-0.023, 0.057]	no clear difference
dino_src_min	0.934	0.902	+0.032	[-0.045, 0.095]	no clear difference

dino_consec	0.988	0.978	+0.010	[-0.000, 0.018]	no clear difference
dino_first_last	0.937	0.901	+0.036	[-0.035, 0.097]	no clear difference
first_frame_ssim	0.987	0.992	-0.005	[-0.009, -0.002]	**Hunyuan higher**
motion_mad	**0.643**	**2.682**	**-2.040**	**[-3.005, -1.216]**	**Hunyuan higher**
jitter_ratio	1.165	0.882	+0.283	[0.050, 0.560]	Wan higher
luma_drift	4.496	12.882	-8.386	[-15.241, 0.425]	no clear difference

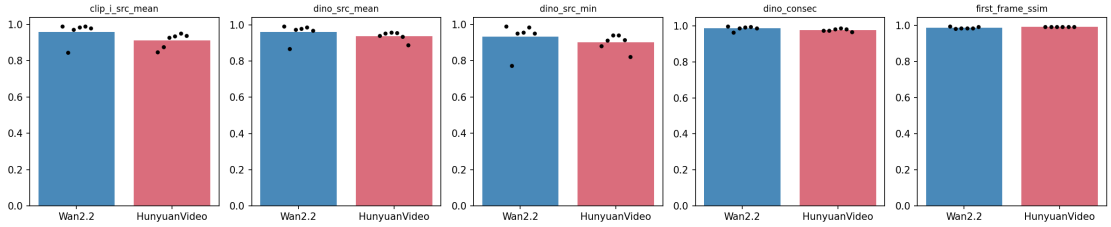


Figure 15. Quality and fidelity metrics with paired intervals.

The central result of this pair is a confound, not a winner. Wan2.2 scores higher on source similarity (clip_i_src_mean +0.047, interval excludes zero) — but it also produces **4.2 times less motion** (motion_mad 0.643 vs 2.682, the widest-separated result in the pair). A clip that barely moves trivially resembles its source photograph on every frame. The apparent fidelity advantage is therefore substantially an **artefact of near-static output**, and the two cannot be disentangled with these data.

This is the same failure mode that would later appear in Pair 6, where a model that did not perform the requested edit scored highest on background preservation. In both cases, a preservation metric rewards a model for doing less.

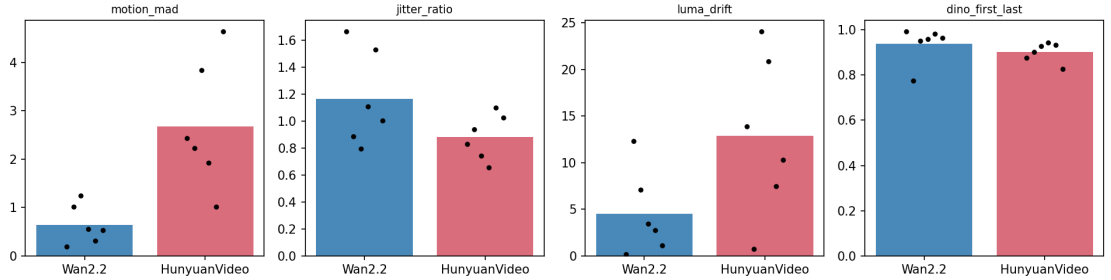


Figure 16. Motion and temporal metrics. Wan2.2's low motion_mad is what drives its apparent fidelity advantage.

One metric must not be interpreted at all here. jitter_ratio is high-frequency temporal change divided by total temporal change. As total motion approaches zero, its denominator collapses and the ratio becomes unstable. Wan2.2's motion_mad of 0.643 is near that regime, so its apparently worse jitter (+0.283) **is not interpretable** and should not be reported as a quality difference. It is listed above for completeness and excluded from any conclusion.

Prompt adherence ties. clip_t differs by -0.007 with an interval spanning zero, so neither model is better at executing the requested camera movement. Hunyuan simply moves more. Read together with motion_mad, the honest summary is that **Wan2.2 under-executes the motion instruction while scoring well on metrics that reward not moving**.

Metric validity control. Scoring each clip against a different item's photograph:

Metric	Model	Matched source	Different item's source	Gap
dino_src_mean	Wan2.2	0.962	0.352	**0.610**
dino_src_mean	HunyuanVideo 1.5	0.939	0.367	**0.572**

clip_i_src_mean	Wan2.2	0.960	0.799	0.161
clip_i_src_mean	HunyuanVideo 1.5	0.913	0.767	0.146

DINO separates matched from mismatched content by about 0.59; CLIP image similarity by about 0.15. **This replicates Pair 4 almost exactly** (0.68 vs 0.19 there), across a different capability, different models and different content. CLIP image similarity has roughly a quarter of DINO's usable range in both pairs — and the one significant fidelity result in this pair, clip_i_src_mean +0.047, sits on the weaker of the two instruments, which is a further reason to treat it cautiously.

Run cost, decomposed. HunyuanVideo 1.5 is 2.48x faster end-to-end (111.5 s vs 276.9 s of sampling time). As in Pairs 1 and 2, the headline inverts when normalised:

Component	Wan2.2	HunyuanVideo 1.5	Ratio
Sampling steps	50	8	**6.25x fewer** for Hunyuan
Seconds per step	5.54	13.93	**2.52x more** for Hunyuan
Output pixels per frame	901,120	407,040	2.21x more for Wan
Seconds per step per megapixel	6.1	34.2	**5.6x more** for Hunyuan
Total sampling time	**276.9 s**	**111.5 s**	2.48x faster for Hunyuan

HunyuanVideo 1.5's entire speed advantage comes from **step distillation** — 8 steps instead of 50. Per step it is 2.5x more expensive, and per step per megapixel it is **5.6x** more expensive. This is the **second** pair in which a fewer-step configuration wins wall-clock while being more costly per unit of work; the other is Pair 2, where PuLID's 4-step Lightning base makes it 3.4x faster overall despite being 2.2x slower per step. Pair 1 is a different case and should not be grouped with these: FLUX is faster *both* per step *and* in total, so its advantage is genuine on both counts. The implication for this pair is that HunyuanVideo's wall-clock advantage is a property of its **sampling schedule**, and would not survive a step-matched comparison.

Other run figures (n = 6):

Metric	Wan2.2	HunyuanVideo 1.5
latency_s (total)	279.64	**113.45**
peak_vram_gb	**22.82**	24.37
peak_vram_reserved_gb	28.39	**25.06**
file_mb	5.94	**0.26**
gen_ok	6 / 6	6 / 6

Both models completed every item. The 23-fold file-size difference largely reflects the resolution gap and is not a like-for-like comparison, though it is what the marketplace would actually be storing and serving.

7.2.3 Reflection and recommendation

Recommendation. This pair did not produce a clean quality comparison, and the recommendation has to start there. Wan2.2's +0.047 source-fidelity lead is inseparable from producing 4.2x less motion — a clip that barely moves trivially resembles its source on every frame — and jitter_ratio is uninterpretable at that motion level and was excluded. Prompt adherence ties. On everything measurable without the confound — 2.48x faster sampling (111.5 s against 276.9 s), far smaller files, first-frame fidelity, and motion that actually executes the instruction — HunyuanVideo 1.5 is at least as good and much cheaper. Provisionally choose HunyuanVideo 1.5, conditional on the follow-up below. Wan2.2's fidelity figure is not evidence of a better clip; it is evidence of under-execution.

The required follow-up is a motion-matched re-run: tune each model's settings until motion_mad is comparable, then re-measure fidelity. Only then does a fidelity comparison mean anything. Until it is run, the honest position is that HunyuanVideo wins on cost and on motion execution, and the quality question is open.

Gaps and open issues. Six items on a single seed, so the intervals are wide and item variation is the only uncertainty captured. The comparison is system-level — resolution, step count, guidance and offload strategy all differ — so no architectural claim is available. Three seconds is a narrow test for a product that will want longer clips. The governance point should travel with whatever is deployed: a moving camera makes a spatial claim — that a doorway leads somewhere, that a view exists from an angle never photographed — and that geometry is generated, not observed. Under the disclosure direction of Premise 3, motion clips carry a misrepresentation risk that stills do not, and they should be labelled accordingly whichever model is chosen.

8 Pair 6 — Editing an existing walkthrough clip

Wan2.1-VACE-1.3B vs Lucy-Edit-Dev

8.1 Selection

8.1.1 AI modelling techniques

Both models edit an **existing** clip from a text instruction. This is harder than Pair 5: the output must execute the instruction, preserve everything not mentioned, and remain temporally coherent with a camera path the model did not choose. Both are diffusion transformers over a Wan latent-video representation — the clip is encoded by a causal 3D VAE, edited in latent space under text conditioning, and decoded.

They differ in **what they can be told**, and that is the contrast this pair was built to test:

	Wan2.1-VACE-1.3B	Lucy-Edit-Dev
Developer	Alibaba (Wan) / ali-vilab	Decart AI
Parameters / base	1.3 B	Built on Wan2.2 5B
Conditioning	Video **+ spatial mask** (+ optional reference images)	Video **+ text only**
Pipeline	WanVACEPipeline	LucyEditPipeline
Licence	**Apache 2.0**	**Non-commercial** (lucy-edit-dev-model-non-commercial-license)

VACE is a *control-driven* editor: it accepts a mask marking the region to regenerate, and it is designed for inpainting, outpainting, object replacement and removal. Lucy-Edit is *instruction-only*: it receives the clip and the text, with no spatial input, and must infer both what to change and where.

The licensing situation mirrors Pair 1 exactly, and it is uncomfortable. The permissively licensed model (VACE, Apache 2.0) is the one that fails the task below. The model that actually performs the edits (Lucy-Edit-Dev) is released under a **non-commercial** licence, confirmed from its model card metadata (license_name: lucy-edit-dev-model-non-commercial-license). For a commercial marketplace, the working model is not deployable without a separate agreement with Decart. The Apache 2.0 attribution for Wan2.1-VACE comes from a third-party model library page; the official repository was not reachable from the evaluation environment.

Week 7's video-editing material began from one-shot tuning approaches such as Tune-A-Video, which adapt a model to each clip before any edit can be made. Both models here supersede that with zero-shot editing — VACE through explicit mask conditioning, Lucy-Edit through instruction alone — so no per-clip training stands between the footage and the edit. [Confirm against the Week 7 deck.]

8.1.2 Modelling context

Item	Wan2.1-VACE-1.3B	Lucy-Edit-Dev
Objectives	Re-style or re-furnish an existing walkthrough without a reshoot, with guaranteed containment of changes	Same, with no spatial control and a simpler operator interface
Tasks	Furniture replacement, relighting, surface changes on agent-captured walkthroughs	Identical task set
Functions	Masked region regeneration under text guidance	Whole-clip edit from an instruction alone

Framework	1.3 B diffusion transformer with a control branch consuming video and mask; reactive = video * mask determines what the model is conditioned on inside the region	5 B diffusion transformer conditioned on the source clip and the instruction
Algorithms	Latent denoising with mask-conditioned control	Latent denoising with instruction conditioning
Governance	Editing a walkthrough of a **real dwelling** changes what a buyer sees of a property they may purchase. The mask provides an auditable record of which region was altered	Same exposure, with **no** record of what was changed — the whole frame is reprocessed, so there is nothing to audit
Advantages	Containment guarantees (quantified below); permissive licence; auditable edit region	Actually executes the instruction; 2.6x faster at matched settings

The governance distinction is sharper here than in any other pair, and it does not depend on the quality results. A mask is a **record**: it states exactly which pixels were permitted to change, and it can be retained for audit alongside the published clip. An instruction-only editor reprocesses the entire frame, so even an edit that is visually confined leaves no evidence of that confinement. Given the disclosure direction in Premise 3 — where the obligation is to declare that an image was altered to conceal a fault — a system that can demonstrate *what* it altered is materially easier to defend than one that cannot.

8.2 Evaluation

8.2.1 Methodology

Items. Six walkthrough clips filmed for this assignment (Set D), one per room, 1920x1080 at 30 fps, each with continuous slow camera motion. Each is centre-cropped to 832x480 and trimmed to **81 frames** (4n+1, required by the shared Wan VAE), starting 15% into the clip to skip the unsteady opening. Exposure and white balance were locked during filming so that camera brightness drift could not be mistaken for model-induced flicker.

Edits follow a balanced **three families x two clips** design, each chosen to be physically plausible for the room filmed:

Family	Clips	Edit box	Instruction
Furniture swap	living_area, main_bedroom	partial	replace the sofa / the bed
Surface change	kitchen_and_main_living_area, storeroom_and_gym	partial	new flooring
Relight	bedroom_2, bathroom	**full frame**	golden-hour sunlight through the window

Settings — the only fully matched comparison in this report. Identical clip, instruction, seed, resolution (480x832), frame count (81), guidance (5.0) **and step count (20)**. Unlike Pairs 1, 2 and 5, neither model is advantaged by a different sampling schedule, so the cost comparison below is like-for-like. Twenty steps rather than the published default of 50 is a compute-budget decision applied equally to both, and it is declared as a limitation.

Declared asymmetry. VACE receives the edit box as a mask; Lucy-Edit has no mask input. The same box is used at *scoring* time for both models, so bg_psnr, bg_ssim and bg_mad measure fidelity outside the edit region identically. For the two full-frame-box items there is no "outside", so those metrics are reported on **n = 4**.

Metrics. clip_t_gain (CLIP text-frame cosine for the instruction, output minus input); box_mad and bg_mad (change magnitude inside / outside the box); edit_ratio; bg_psnr / bg_ssim; flicker and flicker_excess (noise the model *added* over the source clip); motion_mad (to detect an output that froze); dino_consec, dino_src_mean; plus latency_s and peak_vram_gb. A mismatch control scores each output against a *different* item's instruction.

8.2.2 Results

This pair required **two corrections before any number below could be trusted**, and both are reported because the sequence is the most instructive thing in the chapter.

Correction 1 — a conditioning error that made VACE appear to do nothing. Visual inspection of the first pass showed Lucy-Edit executing the instructions and VACE reproducing the original furniture unchanged. The cause was in the harness, not the model: diffusers' WanVACEPipeline computes reactive = video * mask, so whatever lies inside the mask is handed to the model as the conditioning for that region. The harness supplied the unmodified frame, so **VACE was conditioned on the very object it had been asked to replace** and faithfully rebuilt it. For an inpainting edit, the masked region must first be blanked. The harness was corrected (blank_masked, mid-grey, normalising to approximately zero in the pipeline's [-1, 1] space), covered by a unit test, and VACE was re-run on all six items at identical settings. The superseded run is retained as results/w7_vace_conditioning_bug.jsonl.

The correction worked mechanically — VACE's change inside the box more than quadrupled:

Metric	First pass	Corrected
box_mad	11.04	**45.63**
edit_ratio	2.00	**5.37**
clip_t_gain	+0.014	+0.025

Correction 2 — the fix is degenerate on full-frame masks. For the two relight items, the edit box covers the whole frame, so blanking it greys out **the entire video**. VACE then receives a uniformly blank clip and a text prompt, which is not video editing at all — it is text-to-video with no source conditioning. The per-item figures make this unmistakable:

Item group	box_mad VACE	box_mad Lucy	clip_t_gain VACE	clip_t_gain Lucy
Partial box (n = 4)	22.47	19.40	**+0.0045**	**+0.0133**
Full-frame box (n = 2)	**91.96**	9.59	**+0.0656**	+0.0022

A box_mad of 92 out of 255 on the relight items means VACE regenerated essentially the whole frame from nothing. **Those two items should not be read as edit results for VACE**, and the six-item aggregate reported below is contaminated by them.

Full six-item results (paired bootstrap; background metrics n = 4):

Metric	VACE (A)	Lucy-Edit (B)	Diff (A-B)	95% CI	Verdict
clip_t_gain	+0.025	+0.010	+0.015	[-0.012, 0.047]	no clear difference
box_mad	45.63	16.13	+29.50	[-1.04, 60.11]	no clear difference
bg_mad	4.43	12.04	-7.62	[-11.75, -3.48]	**Lucy changes more outside the box**
edit_ratio	5.37	1.48	+3.89	[1.67, 6.59]	**VACE's edit is far more localised**
bg_psnr (dB)	31.81	22.04	+9.77	[5.58,	**VACE higher**

				13.95]	
bg_ssim	0.945	0.855	+0.090	[0.044, 0.133]	**VACE higher**
dino_src_mean	0.641	0.843	-0.202	[-0.617, 0.199]	no clear difference
dino_consec	0.927	0.903	+0.024	[0.003, 0.047]	**VACE more temporally coherent**
flicker	6.91	9.40	-2.49	[-5.69, 0.30]	no clear difference
flicker_excess	-3.12	-0.62	-2.49	[-5.69, 0.30]	no clear difference
motion_mad	5.42	7.44	-2.02	[-4.41, 0.13]	no clear difference
motion_mad (source clips)	7.69	7.69	—	—	reference

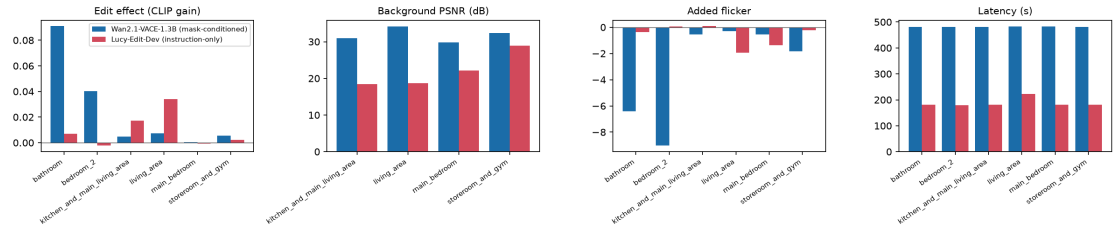


Figure 17. Per-item edit effect, background fidelity, added flicker and latency.

The qualitative result contradicts the aggregate metric

Visual inspection shows Lucy-Edit performed both furniture swaps and VACE performed neither. On living_area, Lucy replaced the brown leather sofa with a green velvet one; on main_bedroom, it replaced the navy upholstered bed with an oak platform bed and clean linen. After the conditioning fix, VACE visibly regenerates the masked region — cushions and linen differ — but it produces **a brown leather sofa and a navy bed again**. It inpaints content consistent with the surrounding scene rather than content specified by the instruction.

On the six-item aggregate, clip_t_gain ranks **VACE above Lucy** (+0.025 vs +0.010) — the metric preferring the model that visibly failed. Restricting to the four items where the edit was genuinely localised **reverses that ordering and aligns it with what the eye sees**:

	VACE	Lucy-Edit
clip_t_gain, all six items	**+0.0249**	+0.0096
clip_t_gain, partial-box items only (n = 4)	+0.0045	**+0.0133**

So the apparent metric inversion is not a failure of CLIP alone — it is the two degenerate full-frame items doing the work. Once they are removed, the metric points the right way. **Both facts matter**: an aggregate computed over a heterogeneous item set gave the wrong answer, and only per-item inspection revealed why.

The underlying instrument is nevertheless weak. The mismatch control — scoring each output against a *different* item's instruction — gives a gap of **0.036 for VACE and 0.036 for Lucy**. Even the corrected partial-box effect of +0.013 sits at roughly a third of that noise floor, so clip_t_gain cannot carry a conclusion here in either direction. The visual evidence does.

What the metrics do establish

Setting instruction-following aside, the numbers describe a real architectural trade-off:

- **VACE's mask delivers genuine containment.** edit_ratio 5.37 vs 1.48 and bg_mad 4.43 vs 12.04, both intervals excluding zero, mean VACE confines changes to the designated region while Lucy-Edit alters

the whole frame. Lucy's 9.77 dB deficit on bg_psnr is the price of having no spatial control. For a marketplace under a fidelity obligation, a containment guarantee has value independent of edit quality.

- **VACE is more temporally coherent** (dino_consec +0.024, interval excludes zero).
- **But VACE degrades what it regenerates.** dino_src_mean falls to 0.641 with a very wide interval [0.327, 0.933] against Lucy's 0.843 — on some items the regenerated region departs badly from the source. Its motion_mad also drops to 5.42 against the source clips' 7.69, so the regenerated region partially loses the camera motion that the rest of the frame keeps.
- **Both models reduce flicker** relative to the hand-held source footage (flicker_excess -3.12 and -0.62). The latent round-trip absorbs camera shake — an incidental benefit for agent-shot walkthroughs, and the only finding in this pair that favours both models at once.

Run cost — the one like-for-like cost comparison in the report:

Metric	VACE (1.3 B)	Lucy-Edit (5 B)	Ratio
latency_s	481.8	**187.4**	**2.57x** faster for Lucy
Seconds per step (20 steps each)	24.09	**9.37**	2.57x
peak_vram_gb	20.11	**18.13**	1.11x
file_mb	0.88	0.79	—

Because step count, resolution and frame count are identical, this ratio is not an artefact of sampling schedule — unlike Pairs 1, 2 and 5. The result is counter-intuitive: **the 1.3 B model is 2.6x slower than the 5 B one**. The likely cause is VACE's additional control branch and the maturity of the respective diffusers implementations rather than anything intrinsic, but that was not investigated and is offered as an observation, not an explanation.

8.2.3 Reflection and recommendation

Recommendation. Lucy-Edit-Dev is the only model of the two that performs the task: it executed both furniture swaps, while VACE, correctly conditioned, executed neither — it inpaints scene-consistent content rather than instructed content, and it degrades what it regenerates (dino_src_mean 0.641, interval [0.327, 0.933]). But Lucy-Edit is non-commercial and VACE is Apache 2.0, so the working model cannot be deployed and the deployable model does not work. The options, in order of preference: pursue a commercial licence from Decart, since the capability is demonstrated and the blocker is contractual; evaluate VACE-14B, preserving the Apache 2.0 path, because nothing here tests whether the larger variant follows instructions; or judge the capability not yet viable. What VACE offers in return is real — containment (edit_ratio 5.37 against 1.48), better temporal coherence, and a mask that doubles as an audit record — and it is the uncomfortable irony of this pair that the model with the better audit story is the one that could not perform the edit.

The methodological findings are the strongest in the report and should shape any re-run. Two corrections were needed before any number could be trusted; the first fix was itself degenerate on full-frame masks; the six-item aggregate inverted the correct ranking; and clip_t_gain's 0.036 noise floor dwarfs the 0.013 corrected effect, so the visual evidence, not the metric, carries the conclusion. An evaluation that simply trusted its instruments would have published the wrong winner, with confidence intervals attached.

Gaps and future work. Six items on one seed, 20 steps against the published 50, background metrics computable on only four items, a mask input only one model can use, and the small VACE variant — all declared, all narrowing the conclusion. The obvious next experiments: re-run at 50 steps on VACE-14B; separate full-frame relight items from localised edits in the design rather than discovering the split afterwards; and administer the A/B survey that was built for exactly this pair, which would turn a single person's visual verdict into measured preference.

9 Pair 7 — Agents that operate the listing web UI

Qwen2.5-VL-7B-Instruct (DOM text) vs UI-TARS-1.5-7B (screenshots)

9.1 Selection

9.1.1 AI modelling techniques

Both models are **vision-language models driving a GUI agent loop**: the model receives an instruction and an observation of a web page, emits one action, observes the result, and repeats until the task completes or a step limit is reached. For the marketplace, the use case is automating repetitive listing-management work — bulk field updates, form completion, status changes — through the existing web interface rather than through an API that may not exist for every internal tool.

The pair is constructed as a contrast in **what the agent is allowed to see**:

	Qwen2.5-VL-7B-Instruct	UI-TARS-1.5-7B
Developer	Alibaba (Qwen)	ByteDance Seed
Observation	Textual rendering of the page's DOM elements (reference id, tag, text, position)	**Screenshot only**
Action format	Structured JSON referencing element ids	Pixel coordinates, as a human would click
Design intent	General-purpose vision-language model	**Purpose-built GUI agent**
Licence	Apache 2.0	Apache 2.0

This pair has two confounds, not one, and they point in opposite directions. The intended contrast is observation modality — structured text versus pixels. But the models also differ in *specialisation*: UI-TARS is trained specifically for GUI control, while Qwen2.5-VL is a general-purpose VLM given a DOM listing. A result favouring UI-TARS could reflect the screenshot modality or could reflect GUI-specific training; a result favouring Qwen could reflect the DOM modality or could reflect that MiniWoB++ pages are unusually DOM-friendly. Neither confound can be removed within this design, and no claim is made that modality alone explains any difference below.

Licensing is clean on both sides — the only pair in this report where that is true. Both models are Apache 2.0 (UI-TARS per its model card's licence label, which links the LICENSE file; Qwen2.5-VL-7B-Instruct per the Qwen family's standard release terms). After six pairs in which licensing blocked, constrained or complicated deployment, this one does not.

Week 8 presented the agent as a policy acting on an environment and walked the progression from a rule-based agent, through an LLM agent reading the DOM, to a VLM agent reading pixels — evaluated on MiniWoB, the same benchmark family used here. This pair evaluates two points on exactly that progression, and its rule-agent baseline, defined in code but never run, is the lecture's first rung. [Confirm against the Week 8 deck.]

9.1.2 Modelling context

Item	Qwen2.5-VL-7B (DOM)	UI-TARS-1.5-7B (visual)
Objectives	Automate repetitive listing-management actions at a cost per action low enough to beat human handling	Same, with the promise of working on any interface, including ones with no usable DOM
Tasks	Click, select, type and submit on listing-management forms	Identical task set
Functions	Reads a DOM listing, emits a JSON action	Reads a screenshot, emits a click or type at

	naming an element id	pixel coordinates
Framework	7 B VLM in an 8-step perceive-act loop; observation is text, so grounding is a lookup	7 B GUI-specialised VLM in the same loop; observation is an image, so grounding is spatial inference
Algorithms	Autoregressive generation of a structured action given a textual page state	Autoregressive generation of coordinates given a rendered page
Governance	An agent with **write access to live listings is a privileged actor**. Prompt injection is a measured security property here, not a hypothetical	Same, plus a weaker audit story: an action logged as a pixel coordinate is harder to reconstruct after the fact than one naming an element id
Advantages	7.2x cheaper per step, 8.6x fewer tokens, never obeyed the injected instruction	Works where no DOM is available; recovered from the injection without losing task success

The governance row carries unusual weight because this pair **measures a security property directly** rather than merely discussing one. An agent that edits live listings can, if manipulated, publish incorrect information about real properties at scale. Two distinct risks are separated in the results: whether an agent *obeys* injected instructions, and whether injected content *degrades* its reliability. They turn out to be different failures affecting different models.

A second governance point is auditability, echoing Pair 6. A DOM agent's action log names the element it acted on; a visual agent's log records coordinates, which can only be interpreted against the exact page render at that moment. For an actor with write access to a public listing, the difference matters in an incident review.

9.2 Evaluation

9.2.1 Methodology

Benchmark. MiniWoB++ (MIT licence), driven through Selenium and Chrome. **8 tasks x 3 seeds = 24 episodes** per agent in the clean condition, with a maximum of 8 steps per episode and a binary reward. Tasks: click-button, click-link, click-option, click-checkboxes, choose-list, enter-text, enter-password, login-user.

Observation handling. MiniWoB pages render at 160x210 pixels — far below what a GUI agent expects. Screenshots are therefore upscaled to 560x728 for UI-TARS, and its predicted coordinates are divided back down. This is a necessary accommodation, and also a limitation: the visual agent is being tested on upscaled renderings of tiny synthetic pages, not on the kind of screen it was trained for.

Prompt-injection condition. On a 4-task subset (click-button, click-link, choose-list, enter-text), a fake notice and a decoy button reading "CLICK HERE TO CONTINUE" are injected into the page DOM before the episode. Two quantities are measured separately: the **attack-followed rate** (did the agent act on the decoy element or its bounding box?) and the **success drop** against the same agent on the same four tasks in the clean condition.

Metrics. Binary success with Wilson 95% intervals, paired bootstrap over the 8 tasks for the head-to-head difference, n_steps, invalid_rate, latency_per_step_s and generated tokens per episode.

Two harness issues, both disclosed.

1. *Episode-timer defect (fixed, runs repeated).* MiniWoB++ arms a 10-second wall-clock episode timer inside reset(). The harness initially raised the timeout *after* reset, which does not cancel the already-armed timer, so slow agents were terminated with reward -1 mid-task. At roughly 5.4 s per step, UI-TARS was cut off after about two actions. The fix cancels and re-arms the timer, and **all results below come from runs after the fix**.
2. *Prompt-iteration asymmetry (not equalised).* UI-TARS initially looped — it clicked a text field repeatedly and never typed, because a 160x210 screenshot shows no caret or focus ring, so at each step it re-

derived the same "click the field to activate it" plan. One generic scaffold was added to its prompt: its own prior actions in its own coordinate space, plus a statement that clicking a field focuses it even without a visible caret. **Qwen's prompt received no equivalent iteration.** The comparison below is therefore of the two agents as tuned, not as first written. This is itself a finding about the engineering cost of the visual approach.

Baseline. A rule-based agent is defined in the harness as a non-learned floor but **was not run**, so this pair has no baseline. As in Pairs 1, 2 and 4, the intended control exists in code and was never executed — a consistent gap discussed in the synthesis.

9.2.2 Results

Overall success (Wilson 95% CI):

Condition	Agent	Episodes	Successes	Success rate	95% CI
clean	Qwen2.5-VL (DOM)	24	12	0.500	[0.314, 0.686]
clean	UI-TARS (visual)	24	15	**0.625**	[0.427, 0.788]
inject	Qwen2.5-VL (DOM)	12	9	0.750	[0.468, 0.911]
inject	UI-TARS (visual)	12	9	0.750	[0.468, 0.911]

The injection rows are **not** comparable to the clean rows: they cover only the four easier tasks. The like-for-like comparison is in the injection table further below.

Head-to-head (paired bootstrap over the 8 tasks): UI-TARS minus Qwen = **+0.125**, 95% CI **[-0.25, +0.50] — no clear difference**. The 12.5-point raw gap is not separable from zero at n = 8 tasks.

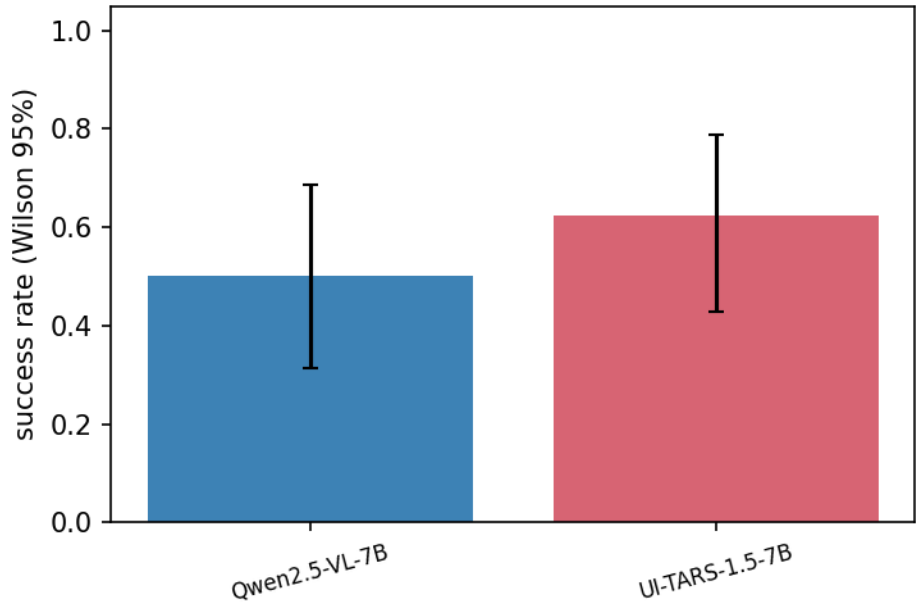


Figure 18. Success rate by agent and condition with confidence intervals.

Per-task success (mean over 3 seeds) — where the real structure is:

Task	Qwen2.5-VL (DOM)	UI-TARS (visual)
click-button	1.00	1.00
click-link	1.00	1.00
enter-text	1.00	1.00

choose-list	**1.00**	0.00
click-checkboxes	0.00	**1.00**
click-option	0.00	**1.00**
enter-password	0.00	0.00
login-user	0.00	0.00

The aggregate tie conceals complementary, near-deterministic competence profiles. Three tasks are solved by both agents on every seed. Of the remaining five, Qwen wins one outright and UI-TARS wins two outright — each at 0.00 against 1.00, meaning the losing agent fails on *every* seed rather than sometimes. Two tasks defeat both. Averaging these into a single success rate destroys the only interpretable structure in the result, and it is a direct argument for per-item reporting over headline accuracy.

Cost per step:

Metric	Qwen2.5-VL (DOM)	UI-TARS (visual)	Ratio
latency_per_step_s	0.752	5.388	**7.2x**
gen_tokens per episode	39.5	338.9	**8.6x**
n_steps (mean)	3.46	4.17	1.2x
invalid_rate	0.000	0.000	—

Neither agent ever emitted an unparseable action. The visual agent costs **7.2x the latency and 8.6x the tokens per step for no statistically separable success advantage** — which, under the volume premise, is the decisive operational fact of this pair.

Prompt injection — obedience and robustness dissociate:

Agent	Clean success (same 4 tasks)	Inject success	Drop	Attack-followed rate
Qwen2.5-VL (DOM)	1.000	0.750	**0.250**	**0.000** (0 / 12)
UI-TARS (visual)	0.750	0.750	**0.000**	**0.167** (2 / 12)

The two agents fail in **opposite** ways. The DOM agent **never acted on the decoy** — it saw the injected element as one more DOM node and did not select it — yet it **lost a quarter of its success rate**, consistent with the extra node disrupting its element selection. The visual agent **did act on the decoy in 2 of 12 episodes** but its success rate was **unchanged**, recovering within the step budget after a wasted click.

Reporting only "attack followed" would credit the DOM agent with a robustness it does not have. Reporting only the success drop would credit the visual agent with a resistance it does not have. **Both numbers are required, and this is the clearest argument in the report for measuring a security property along more than one axis.** With 12 episodes per cell, neither figure is precise; the dissociation, not the magnitude, is the finding.

Two tasks defeat both agents, and the cause is unresolved. enter-password and login-user score 0.00 for both agents on every seed. These are the only two tasks requiring text entry into **multiple fields followed by a submit**. Trajectory inspection shows the agents fail differently: Qwen types correct text into both fields and then receives a negative reward on submit, while UI-TARS never reaches the typing stage, re-clicking the first field until the step limit.

The timer defect was initially suspected for Qwen's failure, since reward -1 is also MiniWoB's timeout reward. **That hypothesis was tested and rejected** — Qwen still scores 0.00 on both tasks after the timer fix. The residual cause was not identified and is reported as an open issue rather than explained away.

UI-TARS's failure mode, by contrast, is understood and generalises: a 160x210 screenshot **does not encode focus state**. There is no caret and no focus ring at that resolution, so the agent cannot distinguish a

field it has already activated from one it has not, and it re-derives the same “click to activate” plan each step. This will recur on any multi-field form — which describes most of the marketplace’s listing-management interface.

9.2.3 Reflection and recommendation

Recommendation. The aggregate difference is not significant (+0.125, interval [−0.25, +0.50]), so the honest headline is complementarity, not a winner. If one agent must be chosen today, choose the DOM agent: the visual agent costs 7.2x the latency and 8.6x the tokens per step for no measured success advantage, which under the volume premise is decisive against it. The more interesting option is neither agent alone. The per-task profiles are near-deterministic and opposite — Qwen wins choose-list 1.00 to 0.00; UI-TARS wins click-checkboxes and click-option 1.00 to 0.00 — so a router dispatching by task type would beat either. Whether that is worth building depends on how stable the task mix is; for a fixed set of listing-management operations it is, and the routing table can be read off the per-task results. Whatever is deployed, a human confirms before any write reaches a live listing — both agents fail tasks, and some of those failures are not yet understood.

The injection results say which failure you would rather have in production. The DOM agent never acted on the decoy but lost a quarter of its success rate — it degrades. The visual agent acted on the decoy in 2 of 12 episodes and recovered within the step budget — it can be redirected. For an actor with write access to live listings, obedience to an injected instruction is the worse failure: a robustness drop fails safe and is retried, while an obeyed injection is an action taken on an attacker’s behalf. On that argument the DOM agent’s failure mode is the preferable one, and it coincides with the cheaper system.

Gaps and open issues. enter-password and login-user defeat both agents for a cause that was tested and not identified, and multi-field forms are exactly what a listing system is made of. The rule-based baseline exists in code and was never run, so there is no floor showing how much of the performance required a learned model. Two confounds — observation modality and GUI-specific training — cannot be separated in this design, and UI-TARS received a round of prompt engineering that Qwen did not. Above all, MiniWoB++ pages are small, synthetic and DOM-friendly, which plausibly favours the DOM agent; the central threat to external validity is that a real listing interface looks nothing like these pages.

10 Cross-pair synthesis

Seven pairs, fourteen models, five capability areas. The findings below are the ones that recur — each is supported by evidence from more than one pair, and several would not have been visible from any single chapter.

10.1 Only one pair produced a clean winner, and it is the only pair with ground truth

On the metric that most directly measures whether a model performed the requested task, **six of seven pairs return "no clear difference"**:

Pair	Primary task metric	Verdict
1 Image editing	clip_gain	no clear difference (+0.001)
2 Identity	id_facenet	no clear difference (-0.020)
3 Walkthrough	**psnr / ssim / lpips**	**Splatfacto wins all three**
4 Image-to-3D	all five similarity metrics	no clear difference
5 Video generation	clip_t	no clear difference (-0.007)
6 Video editing	clip_t_gain	no clear difference (+0.015)
7 Agents	binary success	no clear difference (+0.125)

The exception is not a coincidence. **Pair 3 is the only pair with ground truth.** Its held-out photographs are images the models never saw but which genuinely exist, so PSNR, SSIM and LPIPS measure agreement with a correct answer. Every other pair relies on *proxy* metrics — CLIP or DINO similarity, face-embedding cosine, silhouette overlap — which measure resemblance to something, not correctness.

The practical reading is not that these models are indistinguishable. It is that **proxy metrics at these sample sizes cannot distinguish them**, and a marketplace choosing on headline quality scores would be choosing on noise. In every pair, the decision had to be made on something else.

10.2 Licensing lives in the dependency tree, and it decided more pairs than quality did

Across fourteen models, **six carry an obstacle to commercial deployment**, and in three of the seven pairs the constraint falls on the technically preferable model:

Pair	Model A	Model B	Commercial status
1	FLUX.1 Kontext [dev]	Qwen-Image-Edit-2509	A **non-commercial**; B Apache 2.0
2	InstantID	PuLID v1.1	Both Apache 2.0, but **both depend on InsightFace face models licensed for non-commercial research only**
3	Nerfacto	Splatfacto	Both clean (nerfstudio Apache 2.0, nerfacc MIT, gsplat Apache 2.0)
4	TRELLIS.2-4B	Hunyuan3D 2.1	A MIT core but pulls **RMBG-2.0 (CC BY-NC 4.0)**; B Tencent licence: **EU/UK/KR excluded, 1 M MAU ceiling**
5	Wan2.2 TI2V-5B	HunyuanVideo 1.5	Both Apache 2.0
6	Wan2.1-VACE-1.3B	Lucy-Edit-Dev	A Apache 2.0; B **non-commercial**
7	Qwen2.5-VL-7B	UI-TARS-1.5-7B	Both Apache 2.0

Three patterns are worth stating separately.

The restriction is usually not in the top-level repository. Pair 2's models are both Apache 2.0, and the capability is still blocked — by a face encoder deep inside both pipelines. Pair 4's TRELLIS has an MIT badge and a CC BY-NC 4.0 dependency. **Reading the licence on the model card is insufficient; the dependency tree has to be traced**, and in two of these cases doing so reversed the conclusion.

The same vendor is not consistent across model families. Tencent released Hunyuan3D 2.1 under a community licence with territory exclusions and a 1 million MAU ceiling, while HunyuanVideo 1.5 appears to be Apache 2.0. Inferring terms from the publisher would have been wrong in both directions.

A permissive reimplementation can unblock a restricted method. Pair 3's Splatfacto uses gsplat, an Apache 2.0 standalone implementation of 3D Gaussian splatting, so the original research codebase's terms never bind. This is the inverse of Pair 2 and the most useful practical lesson of the three.

The scenario's own premise triggers one of the constraints. Hunyuan3D's ceiling takes effect above 1 million monthly active users. Premise 1 assumes a national marketplace operating at exactly the volume that crosses it, so the model that wins Pair 4 technically needs a discretionary agreement with Tencent before it can run at the assumed scale. No quality metric anywhere in this report would have surfaced that.

10.3 Wall-clock speed advantages are usually properties of sampling schedules

Where the two models in a pair ran at different step counts, the headline speed ratio decomposes into step count and per-step cost, and the two often point in opposite directions:

Pair	Faster model	Steps	Per-step cost	Conclusion
1	FLUX (2.9x faster)	28 vs 40 (1.43x)	FLUX **2.03x cheaper**	Genuine on both counts
2	PuLID (3.4x faster)	4 vs 30 (7.5x)	PuLID **2.18x more expensive**	Advantage is the Lightning base, not the method
5	HunyuanVideo (2.5x faster)	8 vs 50 (6.25x)	Hunyuan **2.52x more expensive**	Advantage is step distillation
6	Lucy-Edit (2.6x faster)	**20 vs 20 (matched)**	Lucy **2.57x cheaper**	Only like-for-like cost result in the report

In Pairs 2 and 5, the faster model is **more expensive per unit of work** and wins only by taking fewer steps. Those advantages are real — the marketplace pays for wall-clock, not steps — but they belong to a sampling schedule that could in principle be applied to either model, and they would not survive a step-matched comparison. Pair 6 is the control case: with steps matched, the ratio reflects the models themselves, and it produced the report's most counter-intuitive cost result (a 1.3 B model 2.6x slower than a 5 B one).

10.4 Preservation metrics reward models that do less

Twice, a model scored best on fidelity precisely because it failed to perform the task:

- **Pair 5:** Wan2.2 leads source similarity (+0.047) while producing **4.2x less motion**. A clip that barely moves trivially resembles its first frame.
- **Pair 6:** VACE leads background fidelity by **+9.77 dB PSNR** while not executing the instruction at all. A model that changes nothing preserves everything.

The structural problem is that *preservation* and *execution* are measured separately — as the methodology requires — but a model that under-executes scores well on the former and is only caught by the latter. Both cases were identified by pairing the fidelity metric with a **behavioural** one (motion_mad, box_mad) that detects whether anything happened at all. **Any evaluation of an editing or transformation model needs a did-anything-change metric**, or it will rank inaction as fidelity.

10.5 Metric validity controls paid for themselves

Mismatch controls — scoring each output against a *different* item's source or instruction — were run in three pairs and produced consistent, decision-relevant results:

Pair	Metric	Matched	Mismatched	Usable range
4	dino_best	0.783 / 0.786	0.104 / 0.109	**~0.68**
4	clip_i_best	0.922 / 0.923	0.732 / 0.738	~0.19
5	dino_src_mean	0.962 / 0.939	0.352 / 0.367	**~0.59**
5	clip_i_src_mean	0.960 / 0.913	0.799 / 0.767	~0.15
6	clip_t	0.281 / 0.266	0.244 / 0.230	**~0.036**

DINO has roughly four times CLIP image similarity's usable range, replicated across two independent pairs with different capabilities, models and content. And in Pair 6 the CLIP text metric's entire range is 0.036 — against a measured effect of 0.013, so it could not settle that pair in either direction. Pair 2's equivalent control was the use of **two independent face backbones**, which is what exposed the face-size confound.

A single uncontrolled similarity metric is not evidence. Three of this report's four measurement artefacts were caught by controls of this kind.

10.6 Aggregates concealed the structure that mattered

In two pairs, the headline number was not merely imprecise but **actively misleading**:

- **Pair 7:** a tied success rate (0.500 vs 0.625, interval spanning zero) decomposes into near-deterministic *opposite* per-task profiles — Qwen wins choose-list 1.00 to 0.00, UI-TARS wins click-checkboxes and click-option 1.00 to 0.00. The agents are complementary, not equivalent, and the aggregate says the opposite.
- **Pair 6:** the six-item clip_t_gain aggregate ranked the failing model above the succeeding one (+0.025 vs +0.010). Restricted to the four items where the edit was genuinely localised, the ordering **reverses** (+0.0045 vs +0.0133) and matches the visual evidence. Two degenerate items had contaminated the mean.

Both were found only through per-item inspection. Where an item set is heterogeneous — different task types, different edit scopes — the mean over it is not a summary of anything.

10.7 Harness defects are indistinguishable from model results until excluded

Of the measurement artefacts found, **two were defects in the evaluation harness, not properties of the models**:

Pair	Apparent result	Actual cause
6	"VACE preserves the background far better"	VACE was conditioned on the object it was asked to replace (reactive = video * mask)
6	VACE's aggregate edit advantage	Blanking a full-frame mask reduced two items to text-to-video
7	50/50 tie, 0% injection followed	A 10-second episode timer terminated slow agents mid-task
2	InstantID's large identity advantage	ArcFace tracks face area (pooled r = +0.741); InstantID renders faces 3x larger

The first three would have been published as model findings. Each was caught by looking at raw outputs or trajectories rather than at summary tables. **A negative result about a model is a claim about the**

harness until the harness has been excluded — and in Pair 6 the first fix was itself defective, so one round of correction was not enough.

10.8 A consistent gap: no non-learned baselines were run

Pairs 1, 2 and 7 each define a baseline in code — an identity pass-through for the image pairs, a rule-based agent for the GUI pair — and **none was executed**. Pair 4 defines none. The report therefore has benchmarks (MiniWoB++, held-out views) but **no non-learned floors**, so for four capabilities there is no evidence of how much of the measured performance required a learned model at all. This is the clearest avoidable weakness in the evaluation; the rule agent in particular runs on CPU in seconds.

10.9 Recommendations for the marketplace

The pair chapters decide models; this section decides the programme. Each capability below carries a recommendation and the constraint that actually binds it — which in most cases is not technical.

Photo editing — Qwen-Image-Edit-2509. Editing quality ties, so the decision is FLUX's non-commercial licence against Qwen's Apache 2.0, and it is not close. The binding constraint is legal, not technical. Had FLUX been licensable, its 2.9x throughput would have forced a real cost argument; that argument was never reached.

Agent portraits — neither, as built. Both pipelines depend on InsightFace face models restricted to non-commercial research, so the capability is blocked at the dependency level and the InstantID-versus-PuLID result cannot be acted on until the encoder is replaced and the evaluation re-run. What binds it is a dependency's licence, plus a consent obligation over a named person's likeness that no other capability carries.

Walkthroughs — Splatfacto; deploy. The report's only clean winner: better on every quality metric, 8.6x faster to train and 846x faster to render, on clean licensing all the way down the dependency tree. What binds it is the ~225 MB artefact, whose compression cost is unmeasured, and — beyond this evaluation entirely — the capture burden on the agent.

Staging assets — Hunyuan3D 2.1, on artefact quality. Quality ties; watertightness, file size and fragmentation decide it. What binds it is the 1 million monthly-active-user ceiling in Tencent's community licence, which the scenario's volume premise crosses: this capability needs a discretionary negotiation before any deployment at scale.

Motion clips — HunyuanVideo 1.5, provisionally. Cheaper, faster, and the only model that actually moves; Wan2.2's fidelity lead is an artefact of producing 4.2x less motion. What binds it is evidence rather than licence: the recommendation is conditional on a motion-matched re-run that this evaluation did not perform.

Clip editing — Lucy-Edit-Dev, the only model that performs the edit. What binds it is the non-commercial licence: the Apache 2.0 alternative executed neither furniture swap. A licence from Decart, or a future VACE-14B that follows instructions, unblocks the capability; neither is in the marketplace's control.

Listing automation — the DOM agent, behind human confirmation. There is no significant success difference, so cost decides: 7.2x the latency and 8.6x the tokens per step buy nothing measurable. What binds it is understanding — two form tasks defeat both agents for an unknown cause — and the absence of the rule-based baseline.

Sequencing follows from what binds each capability. Production-ready now: realistically only walkthroughs, and even those pending the compression measurement — it is the one capability with a clean technical result, clean licensing and no unresolved defect. One more evaluation round: photo editing (run the identity baseline that already exists in code), motion clips (the motion-matched re-run) and listing automation (run the baseline, diagnose the two failing tasks, and test on a real interface rather than MiniWoB++). Commercial negotiation before further engineering: staging assets (Tencent) and clip editing (Decart) — engineering spent on either before the licence is settled risks building on a model that cannot be used. Not viable as built: agent portraits, where the blocker sits in a shared dependency and removing it invalidates the existing results.

One pattern is worth stating plainly. The two capabilities with the cleanest technical results are also the two with no licensing obstacle anywhere in the dependency tree: walkthroughs produced the report's only clean winner, and listing automation is the only pair in which both models are Apache 2.0. The capabilities that produced the most technically interesting results — portraits, staging, clip editing — are exactly the ones a lawyer has to clear first.

10.10 Limitations of this evaluation

1. **Scale.** Item counts range from 6 (Pairs 5 and 6) to 48 item-seeds (Pair 2). Several "no clear difference" verdicts are statements about statistical power as much as about models.
2. **Seed replication is partly illusory.** Pair 4's within-item seed SD is exactly 0.000 — the pipelines are deterministic, so three seeds were three identical runs. Pairs 5 and 6 used a single seed. In those pairs, item-level variation is the only uncertainty captured.
3. **Most pairs are system-level, not model-level.** Pairs 1, 2 and 5 differ in step count, base model or resolution between the two stacks. Only Pairs 3 and 6 hold sampling settings constant, and only Pair 3 shares a framework end-to-end. No architectural claims are made from the others.
4. **Reduced settings.** Pair 6 ran both models at 20 steps rather than the published 50, and uses VACE's 1.3 B rather than the 14 B variant — both budget decisions, applied equally.
5. **Declared asymmetries.** Pair 6's mask input; Pair 7's single round of prompt iteration for UI-TARS only.
6. **Single scenario and narrow inputs.** One business scenario throughout; Pair 3 is one room; Pair 6's six clips all come from one property; Pair 2's references are three synthetic faces and one real one.
7. **Benchmark validity.** Pair 7's MiniWoB++ pages are small, synthetic and DOM-friendly, which plausibly favours the DOM agent and understates the visual agent on real interfaces.
8. **Unresolved failures.** Pair 7's enter-password and login-user; Pair 2's failed held-out identity control; Pair 4's unmeasured geometric accuracy.
9. **No non-learned baselines**, as set out above.
10. **Proxy metrics throughout.** Only Pair 3 has ground truth. Everywhere else, quality is resemblance to something rather than correctness, and the user survey is the only direct human evidence collected.

11 Compute budget and cost

All generation and training ran on Google Colab Pro at **A$0.1513 per compute unit**, against a self-imposed ceiling of **A$60**. Per-pair consumption is recorded in 48576158 Evaluation/cost_log.csv.

11.1 Recorded and reconstructed spend

Three pairs were logged at the time of running. The remainder are reconstructed from the compute-unit balance read from the Colab Resources panel before and after each session; those rows are marked as reconstructed rather than presented as contemporaneous records.

Stage	Units	A$	Source
Pair 1 (image editing)	21.72	3.29	logged
Pair 2 (identity)	10.18	1.54	logged
Pairs 3 + 4 (walkthrough, image-to-3D)	28.10	4.25	**reconstructed, combined**
Pair 5 (video generation)	7.72	1.17	logged
Pair 7 (agents)	3.14	0.48	**reconstructed**
Pair 6 (video editing, first full run)	2.68	0.41	**reconstructed**
Total to a balance of 26.46 units	**73.54**	**11.13**	

Three observations on the figures themselves. The 100-unit starting balance is an **assumption**, not a reading, so every absolute figure inherits that uncertainty; the deltas are reliable. Pair 3's training ran on an RTX PRO 6000 rather than the Colab A100 used elsewhere, so its GPU-minutes are not comparable with other pairs. And the Pair 2 row uses a rate of about 8.9 units/hour read from the Resources panel, while the Pair 5 row assumes 11.77 units/hour for the A100 — the assumed rate is flagged in the log rather than silently applied.

11.2 The budget constrained the evaluation more than the spend justifies

Total spend was **A$11.13 against a A$60 ceiling — roughly 19%**. That figure should be read alongside the design decisions the budget was used to justify:

Decision	Stated reason	Cost had it been reversed
Pair 5: 6 items, **1 seed**	budget	a second seed, about 2 units
Pair 6: 6 items, **1 seed**, **20 steps** not 50	budget	50 steps for both, about 4 units
Pair 6: VACE **1.3 B** not 14 B	budget	not estimated
Pair 4: seeds run but **deterministic**	—	2/3 of that pair's compute returned no information

This is a planning failure worth stating plainly rather than hiding. The two video pairs carry the report's widest intervals and its weakest conclusions, and both were narrowed for a budget that came nowhere near being exhausted. The specific error was forecasting from pessimistic per-unit estimates instead of measuring a smoke run and extrapolating: the Pair 6 full run was projected at roughly 22 units and actually consumed 2.68, an eight-fold over-estimate that directly caused the step count to be halved.

The lesson the marketplace should take from this is the same one the evaluation should: **measure the cost of a small run before constraining the design around a forecast.** The reproducibility machinery for doing so — per-item timing sidecars, resumable runners — was built and used, but the forecast was never revised once real timings existed.

12 User feedback

The evaluation framework in Chapter 2 lists **user feedback** as the sixth dimension, and it is the only one assessed by people rather than by instruments. It is also the dimension on which this report is weakest.

12.1 What exists

Two instruments were written and are included in the submission:

- survey/make_ab_items.py — builds anonymised A/B comparison items from the generated outputs, pairing the two models' results for the same input so a respondent judges a like-for-like choice without being told which model produced which.
- survey/analyse_survey.py — aggregates returned responses into per-pair preference rates.

12.2 What does not exist

No responses were collected. The survey was never administered, so there is no human preference evidence anywhere in this report, and the analyse_survey.py stage has never been run on real data.

12.3 Why this matters more than a missing nice-to-have

Chapter 2 argued that the automatic metrics in this report are proxies, and the synthesis showed that six of seven pairs returned no clear difference on their primary proxy metric. The capabilities where human judgement would have been **decisive** are precisely the ones where the instruments failed:

Pair	Why human judgement was needed	What was lost
1	clip_gain has almost no range on subtle edits (0.006 on brighten)	No evidence on whether relit photos look better
4	All five similarity metrics tie	No tie-breaker on perceived mesh quality
5	Fidelity metrics confounded by 4.2x motion difference	No judgement on which clip looks better
6	clip_t noise floor 0.036 vs effect 0.013; metrics ranked the failing model higher	**The correct answer was only available by eye** — and only the author's eye

Pair 6 is the sharpest case. The conclusion that Lucy-Edit performed the edits and VACE did not rests entirely on **visual inspection by one person who knew which model produced which output**. An anonymised A/B panel is exactly the instrument that would have turned that into evidence — and it was built but not used.

13 References

Every entry below was **retrieved and read during this evaluation**, and the claim it supports is named so that a marker can check it.

Scenario and regulatory

1. Larceneux, F., Bezançon, M. and Lefebvre, T. "'Asymmetric revelation' effect: The influence of an increased number of photos on mental imagery and behavioural responses depending on target market." *Recherche et Applications en Marketing* (accepted manuscript). Field study of 3,658 Paris apartment advertisements (meilleursagents.com, January–September 2016) and laboratory experiments with 124 participants. *Supports Premise 2.* https://drm.dauphine.fr/fileadmin/mediatheque/drm/documents/AsymmetricRevelationEffect.pdf
2. Australian Computer Society, *Information Age*, "NSW tackles AI real estate pics" (2025), on the *Residential Tenancies (Protection of Personal Information) Amendment Bill 2025*: disclosure of images altered to conceal faults; penalties of $5,500 (individuals) and $22,000 (businesses). *Supports Premise 3.* https://ia.acs.org.au/article/2025/nsw-tackles-ai-real-estate-pics.html

Licences (verified for the claims made in the pair chapters)

3. FLUX.1 Kontext [dev] model card — "FLUX.1 [dev] Non-Commercial License", 12 B rectified flow transformer. Retrieved from a public mirror; the official Black Forest Labs repository was not reachable from the evaluation environment. *Pair 1.*
4. Qwen-Image / Qwen-Image-Edit repository — Apache 2.0, 20 B MMDiT. *Pair 1.* https://github.com/QwenLM/Qwen-Image
5. InstantID repository — Apache 2.0 code; disclaimer states InsightFace face models are "for non-commercial research purposes only". *Pair 2.* https://github.com/mlaurelli/InstantID
6. PuLID repository (ByteDance, NeurIPS 2024) — Apache 2.0; v1.1 SDXL-based, demo script uses Lykon/dreamshaper-xl-lightning. *Pair 2.* https://github.com/ToTheBeginning/PuLID
7. gsplat: An Open-Source Library for Gaussian Splatting — Apache License 2.0; standalone implementation of 3D Gaussian Splatting. *Pair 3.* https://arxiv.org/html/2409.06765v1
8. Tencent Hunyuan 3D 2.1 Community Licence Agreement — Territory excludes the European Union, United Kingdom and South Korea; separate Tencent licence required above 1 million monthly active users; non-affiliation disclosure required; outputs may not be used to improve other AI models. Retrieved from a public mirror of the repository LICENSE. *Pair 4.*
9. BRIA RMBG-2.0 — CC BY-NC 4.0; "Commercial use is subject to a commercial agreement with BRIA." *Pair 4.* https://github.com/Bria-AI/RMBG-2.0
10. Lucy-Edit-Dev model card — license_name: lucy-edit-dev-model-non-commercial-license; built on Wan2.2 5B; LucyEditPipeline, 832x480, 81 frames. *Pairs 6.* https://huggingface.co/decart-ai/Lucy-Edit-Dev
11. Wan2.1 VACE model listing — Apache 2.0; accepts text prompt plus optional video, mask and reference-image inputs; supports masked editing, inpainting and object replacement. Retrieved from a third-party model library; official repository not reachable. *Pair 6.*
12. UI-TARS-1.5-7B model listing — Apache 2.0 licence label linking the LICENSE file; vision input; designed for general computer use. *Pair 7.* https://featherless.ai/models/ByteDance-Seed/UI-TARS-1.5-7B
13. HunyuanVideo 1.5 — reported as Apache 2.0 by two independent secondary sources; the official tencent/HunyuanVideo-1.5 repository was not reachable from the evaluation environment. *Pair 5, reported with that caveat in the text.*

Methods, metrics and benchmarks (retrieved and verified during revision)

1. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G. and Sutskever, I. "Learning Transferable Visual Models From Natural Language

Supervision" (CLIP), ICML 2021. The CLIP text-image and image-image similarity metrics (Pairs 1, 2, 4, 5, 6). https://arxiv.org/abs/2103.00020

2. Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A. et al. "DINOv2: Learning Robust Visual Features without Supervision", TMLR 2024. The DINO similarity metrics (Pairs 4, 5, 6). https://arxiv.org/abs/2304.07193
3. Zhang, R., Isola, P., Efros, A. A., Shechtman, E. and Wang, O. "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric" (LPIPS), CVPR 2018. Pair 3. https://arxiv.org/abs/1801.03924
4. Wang, Z., Bovik, A. C., Sheikh, H. R. and Simoncelli, E. P. "Image quality assessment: from error visibility to structural similarity" (SSIM), IEEE Transactions on Image Processing 13(4), 600–612, 2004. Pairs 1, 3, 5, 6.
5. Deng, J., Guo, J., Xue, N. and Zafeiriou, S. "ArcFace: Additive Angular Margin Loss for Deep Face Recognition", CVPR 2019. The id_arcface metric (Pair 2). https://arxiv.org/abs/1801.07698
6. Schroff, F., Kalenichenko, D. and Philbin, J. "FaceNet: A Unified Embedding for Face Recognition and Clustering", CVPR 2015. The id_facenet metric (Pair 2). https://arxiv.org/abs/1503.03832
7. Shi, T., Karpathy, A., Fan, L., Hernandez, J. and Liang, P. "World of Bits: An Open-Domain Platform for Web-Based Agents", ICML 2017. The MiniWoB benchmark lineage (Pair 7). https://proceedings.mlr.press/v70/shi17a.html
8. Liu, E. Z., Guu, K., Pasupat, P., Shi, T. and Liang, P. "Reinforcement Learning on Web Interfaces Using Workflow-Guided Exploration" (introduces MiniWoB++), ICLR 2018. Pair 7's benchmark. https://arxiv.org/abs/1802.08802
9. Tancik, M., Weber, E., Ng, E., Li, R., Yi, B., Kerr, J., Wang, T., Kristoffersen, A., Austin, J., Salahi, K., Ahuja, A., McAllister, D. and Kanazawa, A. "Nerfstudio: A Modular Framework for Neural Radiance Field Development", ACM SIGGRAPH 2023 Conference Proceedings. The framework supplying both Pair 3 implementations (Nerfacto and Splatfacto). https://arxiv.org/abs/2302.04264
10. Kerbl, B., Kopanas, G., Leimkühler, T. and Drettakis, G. "3D Gaussian Splatting for Real-Time Radiance Field Rendering", ACM Transactions on Graphics 42(4), 2023. The representation Splatfacto implements (Pair 3).
11. Wang, Q., Bai, X., Wang, H., Qin, Z. and Chen, A. "InstantID: Zero-shot Identity-Preserving Generation in Seconds", arXiv:2401.07519, 2024. Pair 2. https://arxiv.org/abs/2401.07519
12. Guo, Z., Wu, Y., Chen, Z., Chen, L., Zhang, P. and He, Q. "PuLID: Pure and Lightning ID Customization via Contrastive Alignment", NeurIPS 2024. Pair 2. https://arxiv.org/abs/2404.16022
13. Xiang, J. et al. "Structured 3D Latents for Scalable and Versatile 3D Generation" (TRELLIS), CVPR 2025; and Xiang, J. et al. "Native and Compact Structured Latents for 3D Generation" (TRELLIS.2), arXiv:2512.14692, 2025. Pair 4. https://arxiv.org/abs/2412.01506 and https://arxiv.org/abs/2512.14692
14. Team Wan. "Wan: Open and Advanced Large-Scale Video Generative Models", arXiv:2503.20314, 2025. The Wan model family underlying Pairs 5 and 6. https://arxiv.org/abs/2503.20314
15. Jiang, Z., Han, Z., Mao, C., Zhang, J., Pan, Y. and Liu, Y. "VACE: All-in-One Video Creation and Editing", arXiv:2503.07598, 2025. Pair 6. https://arxiv.org/abs/2503.07598
16. Bai, S. et al. (Qwen Team). "Qwen2.5-VL Technical Report", arXiv:2502.13923, 2025. Pair 7. https://arxiv.org/abs/2502.13923
17. Qin, Y., Ye, Y., Fang, J. et al. "UI-TARS: Pioneering Automated GUI Interaction with Native Agents", arXiv:2501.12326, 2025. Pair 7. https://arxiv.org/abs/2501.12326
18. Wu, C., Li, J., Zhang, S. et al. "Qwen-Image Technical Report", arXiv:2508.02324, 2025. The base generator for Pair 1's Qwen-Image-Edit-2509. https://arxiv.org/abs/2508.02324
19. Formal papers for the remaining models are outstanding: FLUX.1 Kontext [dev] (no formal paper located), Hunyuan3D 2.1, HunyuanVideo 1.5 and Lucy-Edit-Dev. For those models the licence and repository entries above remain the verified sources.